



Strathmore University

@iLabAfrica Centre

Applied Machine Learning

From First Principles to Production Systems

Javier Serrano
Emmanuel Lee Bundi

Master in Data Science and Analytics
DSA 8401

Nairobi, 2026

Applied Machine Learning: From First Principles to Production Systems in the African Fintech Ecosystem

Javier Serrano & Emmanuel Lee Bundi
iLabAfrica — Strathmore University, Nairobi, Kenya

Lecture notes for DSA 8401 — Applied Machine Learning,
Master in Data Science and Analytics.

© 2026 The authors. All rights reserved.
Draft edition — for enrolled students. Not for redistribution.

Typeset with $\text{\LaTeX}$ using the `book` class.
Figures produced with `TikZ`; code listings with `listings`.

Contact: jgarcia@strathmore.edu

Version 0.1 — August 3, 2026

Contents

Preface

vii

I Foundations and Classical Machine Learning 1

1 Introduction to Machine Learning 3

1.1	Why This Chapter Comes First	3
1.2	What Is Machine Learning?	4
1.2.1	An operational definition	4
1.2.2	ML inverts the direction of programming	5
1.2.3	When ML earns its keep — and when it does not	5
1.2.4	The formal core: empirical risk minimization	6
1.2.5	The Bayes-optimal reference	8
1.2.6	Notation for the rest of the book	8
1.3	The Machine-Learning Landscape	8
1.3.1	The four learning paradigms	8
1.3.2	Two orthogonal operational axes	10
1.3.3	No free lunch, and what it licenses	12
1.4	Why ML Systems Fail in Production	12
1.4.1	The small green box	12
1.4.2	The notebook-to-production gap, localized	13
1.5	The End-to-End ML Lifecycle	14
1.5.1	From CRISP-DM to a monitoring-driven loop	14
1.5.2	It is a loop, not a waterfall	15
1.6	Framing the Problem and Choosing the Metric	15
1.6.1	Step 1 is where projects are won	15
1.6.2	Regression metrics: how the business pays for errors	16
1.6.3	Classification under imbalance: why accuracy lies	16
1.7	A First Look at the Data	18
1.7.1	The raw material	18
1.7.2	A feature dictionary, declared up front	19
1.7.3	What to interrogate, and why	19
1.8	Data, Splits, and Leakage	20
1.8.1	Splits: the discipline that protects your estimate of $R(f)$	20
1.8.2	Data leakage: the silent killer of fintech models	21
1.9	Baselines, Evaluation, and Generalization	23
1.9.1	Why generalization is hard: the bias–variance decomposition	23
1.9.2	Always ship a baseline first	24
1.9.3	Cross-validation: cheap insurance on the estimate	24

1.10	Presenting and Deciding	25
1.10.1	The decision memo	25
1.10.2	Champion, challenger, and shadow mode	25
1.11	Fairness, Explainability, and Regulation	26
1.11.1	Making “fair” precise enough to test	26
1.11.2	Explainability as a deployment requirement	27
1.12	From Model to System: MLOps and the Agentic Outlook	27
1.12.1	The MLOps spine of this course	27
1.12.2	Where LLMs and agents fit — and where they do not	28
1.13	Chapter Summary	29
	Exercises	31
	Key Terms	32
	Further Reading	32
2	Data Preparation and Feature Engineering	33
2.1	Introduction: From Raw Data to Model-Ready Data	33
2.1.1	The Six Dimensions of Data Quality	34
2.1.2	The Unit of Prediction	35
2.2	Missing Data	35
2.2.1	Rubin’s Taxonomy of Missingness	35
2.2.2	Diagnosis Before Treatment	36
2.2.3	Imputation I: Simple Statistics with Indicators	37
2.2.4	Imputation II: Model-Based Methods	37
2.2.5	A Decision Framework	38
2.3	Outliers and Duplicates	38
2.3.1	Detecting Outliers	39
2.3.2	Treating Outliers, and When Not To	39
2.3.3	Duplicates and Entity Resolution	40
2.4	Scaling and Transformations	40
2.4.1	Why Scale, and With What	40
2.4.2	Taming Skew: Log and Power Transforms	41
2.5	Encoding Categorical Variables	42
2.5.1	Low Cardinality: One-Hot and Ordinal	42
2.5.2	High Cardinality I: Frequency Encoding and the Hashing Trick	42
2.5.3	High Cardinality II: Target Encoding	43
2.6	Feature Engineering	43
2.6.1	Engineered versus Learned Features	44
2.6.2	Domain Features for Mobile-Money Credit Scoring	44
2.6.3	Datetime Features and Cyclical Encoding	44
2.6.4	Interactions, Binning, and Crosses	45
2.6.5	What Makes a Feature Good?	46
2.7	Data Leakage	46
2.7.1	Definition and Why It Is Dangerous	46
2.7.2	Six Common Causes	47
2.7.3	Temporal and Group-Aware Splits	47
2.7.4	A Detection Protocol	48
2.7.5	The Structural Antidote: Pipelines	48
2.8	From Pipelines to Feature Platforms	49
	Assignment 2 (graded, 20 marks)	50
	Exercises	50
	Key Terms	51

Further Reading	51
3 Classical Machine Learning and Rigorous Evaluation	53
3.1 Introduction	53
3.2 The Classical Toolbox	54
3.2.1 Why classical models still run production tabular ML	54
3.2.2 Logistic regression	55
3.2.3 Softmax regression for multiclass problems	56
3.2.4 Maximum-margin classification: support vector machines	56
3.2.5 Neighbours and Bayes: two more baselines	57
3.2.6 Choosing among the classical learners	57
3.3 Beyond Accuracy: Metrics that Match the Problem	58
3.3.1 The accuracy trap	58
3.3.2 The confusion matrix	58
3.3.3 Precision, recall, and the F_β family	59
3.3.4 From metrics to money: cost-sensitive evaluation	59
3.4 Ranking Quality: ROC and Precision–Recall Analysis	60
3.4.1 Scores, not labels: the threshold as a free parameter	60
3.4.2 The ROC curve and AUC	60
3.4.3 Precision–recall curves: the imbalance-honest view	61
3.5 Calibration: Do the Probabilities Mean Anything?	62
3.6 Cross-Validation and Honest Uncertainty	63
3.6.1 Why one split is not enough	63
3.6.2 Cross-validating the pipeline	64
3.6.3 Nested cross-validation for model selection	64
3.6.4 Comparing models honestly	64
3.7 Learning from Imbalanced Data	65
3.7.1 Three families of remedies	65
3.7.2 SMOTE: synthesising minority examples	66
3.7.3 The cardinal sin: resampling before splitting	66
3.8 Case Study: Evaluation Design for the Thin-File Credit Scorer	67
3.9 Chapter Summary	68
 II Deep Learning and Generative AI	 71
 III MLOps and Production Systems	 73
 Appendices	 77
A Capstone Project	77
B Solutions to Exercises	79
Chapter 1	79
Chapter 2	84
Chapter 3	87

Preface

This book collects the lecture notes of **DSA 8401 — Applied Machine Learning**, taught in the Master in Data Science and Analytics at iLabAfrica, Strathmore University. It is written for graduate students who already program and know basic statistics, and who want to go beyond training models in a notebook: the goal throughout is *production-grade* machine learning — data pipelines, rigorous evaluation, deployment, and monitoring.

Three commitments shape every chapter:

1. **An end-to-end view.** Models are treated as components of systems. MLOps (Docker, FastAPI, MLflow, CI/CD) is not an afterthought but the destination of the whole arc.
2. **African context as pedagogy.** Worked examples are anchored in the African technology and financial ecosystem: thin-file mobile-money credit scoring (the running thread of the course), trade-corridor underwriting, clinical pathway modelling, and streaming fraud detection. This is a deliberate instructional choice, not decoration.
3. **Modern architectures.** Transformers, LLMs, RAG pipelines, and an outlook on agentic systems are integrated where they matter for applied work.

How this book maps to the course

Each chapter corresponds to one teaching week (three hours: a 1.5-hour lecture plus a 1.5-hour practical lab). The two MLOps weeks are consolidated into a single chapter, and the capstone project brief lives in Appendix ??.

Chapter	Topic	Course week
1	End-to-End Machine Learning	1
2	Data Preparation & Feature Engineering	2
3	Classical ML & Rigorous Evaluation	3
4	Ensemble Methods & Hyperparameter Tuning	4
5	Unsupervised Learning & Dim. Reduction	5
6	Deep Learning Fundamentals	6
7	Computer Vision with CNNs	7
8	Sequence Models & Time Series	9
9	Transformers, LLMs & RAG	10
10	MLOps: Serving, Tracking & CI/CD	11–12
11	Monitoring, Drift & System Design	13
A	Capstone Project	14–15

Prerequisites and companion material

We assume the Data Mining unit, working Python, and comfort with linear algebra and probability at the level of a first graduate course. The primary textbook companion is [Géron \(2022\)](#); [Bishop and Bishop \(2024\)](#) and [Huyen \(2022\)](#) support the deep-learning and MLOps parts respectively. Each chapter closes with a lab section that mirrors the Jupyter notebook used in the practical session.

Javier Serrano & Emmanuel Lee Bundi
Nairobi, 2026

Part I

Foundations and Classical Machine Learning

Data Preparation and Feature Engineering

Week 2

Learning Objectives

By the end of this chapter, and of the Week 2 session it accompanies, you should be able to:

- Audit a raw operational dataset against the six dimensions of data quality and produce a defensible data-quality report.
- Classify missing data as MCAR, MAR, or MNAR using Rubin’s framework, and select and implement an imputation strategy that matches the missingness mechanism.
- Detect and treat outliers and duplicates without destroying the predictive signal they may carry, including entity resolution across multiple identifiers.
- Choose scaling and power transformations with statistical justification, and encode low- and high-cardinality categorical variables, including leakage-safe target encoding.
- Engineer domain features for behavioural credit scoring, recency–frequency–monetary aggregates, ratios, velocity, and cyclical time encodings, with correct *as-of* temporal semantics.
- Define data leakage formally, enumerate its common causes, detect it with a systematic protocol, and prevent it structurally with `scikit-learn` pipelines and group-aware, time-aware splits.

2.1 Introduction: From Raw Data to Model-Ready Data

Chapter 1¹ closed with a warning: the model is the smallest part of a production machine-learning system. This chapter is about the largest part. Surveys of practitioners consistently find that between 60% and 80% of project effort goes to acquiring, cleaning, reshaping, and enriching data, and the experience of large industrial teams is blunt about why: once a workable model exists, *better features deliver a larger performance gain than better algorithms*. The Facebook advertising team put it on record over a decade ago, having the right features matters more

¹If you are reading this chapter standalone: Chapter 1 framed the machine-learning lifecycle as a monitoring-driven loop and introduced the running case study used throughout this book a thin-file mobile-money credit-scoring system for a Nairobi-based digital lender.

than the choice of model, and Huyen’s survey of modern production practice reaches the same conclusion: the feature list of a deployed model only ever grows (Huyen, 2022).

Deep learning complicates but does not overturn this picture. Neural networks *learn* representations from raw perceptual inputs, pixels, waveforms, token sequences, and we will exploit that ability from ?? onwards. But the operational data that dominates applied machine learning in banking, telecommunications, health, and government is *tabular and relational*: transaction logs, registries, claims tables. For such data, gradient descent cannot conjure a feature the flat table does not contain; someone who understands the domain must build it. That someone is you, and this chapter is your toolbox.

The chapter follows the same arc as the Week 2 lecture. We begin with the anatomy of messy data (Section 2.1.1), then treat the three classical pathologies, missing values (Section 2.2), outliers and duplicates (Section 2.3), before turning to the constructive work: scaling and transformations (Section 2.4), categorical encoding (Section 2.5), and feature engineering proper (Section 2.6). The chapter culminates in the subject that ties all of the above together and that, more than any other, separates a graduate exercise from a production system: *data leakage* (Section 2.7). A closing section (Section 2.8) connects today’s pipelines to the feature stores and training-serving consistency problems we will meet again in ??.

Case Study: Feature Signals from Mobile-Money Statements

Our lender scores *thin-file* applicants: borrowers with no credit-bureau history but with rich mobile-money transaction logs, deposits, agent withdrawals, person-to-person (P2P) transfers, airtime purchases, paybill and till payments. The target is binary, $y \in \{0, 1\}$: default within 90 days on a 30-day micro-loan. The raw inputs are statement exports, agent metadata, device and SIM records, and repayment histories from prior loans. Evidence that such behavioural exhaust genuinely predicts repayment is by now solid: mobile-phone usage features alone approach the discriminatory power of bureau scores for banked populations (Bjorkegren and Grissen, 2020).

The raw statement dump, however, is a swamp. Dates arrive in three formats; amounts are strings like "1,500/-"; GPS coordinates are missing for most feature-phone users; the same customer appears under three SIM cards; one agent’s test transactions record withdrawals of KES 9,999,999; and two columns, innocently named, are written by the collections team *after* a borrower defaults. Every technique in this chapter will be judged against a single question: **would this information exist, in this form, at the moment of scoring?**

2.1.1 The Six Dimensions of Data Quality

“Messy” is not an analytical category. Before touching a dataset, decompose its problems along six standard dimensions (Fig. 2.1); each dimension has its own diagnostics and its own remedies, and conflating them leads to remedies that fix one problem by creating another.

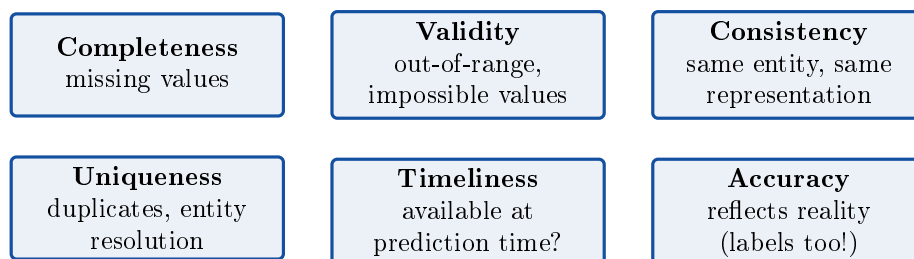


Figure 2.1: The six dimensions of data quality. Each maps to a different diagnostic and a different family of remedies; *timeliness* is the dimension whose violation becomes data leakage (Section 2.7).

In the mobile-money extract, each dimension has a concrete face. A withdrawal of KES 9,999,999 from an account holding KES 12 violates *validity*. The county field containing "Nrb", "Nairobi", and "NBO" for the same place violates *consistency*. One customer holding three SIM cards violates *uniqueness*, and, as we will see in [Section 2.3.3](#), quietly poisons the train/test split. A credit-bureau flag that is updated *after* the default it records violates *timeliness*: the value exists in the warehouse today, but it did not exist at scoring time, and using it constitutes leakage. Finally, *accuracy* applies to labels as much as to features: if the operations team marks a loan “defaulted” when the repayment gateway (rather than the borrower) failed, the model learns to predict gateway outages.

2.1.2 The Unit of Prediction

A subtlety that trips up newcomers immediately: the raw table and the modelling table usually have *different grains*. The statement export has one row per *transaction*, perhaps 400 rows per customer, but the lender scores a *customer at a moment in time*. Everything in this chapter therefore operates in two stages: row-level cleaning on the transaction table, followed by an *aggregation layer* that collapses each customer’s history into a single feature vector aligned to a scoring timestamp t ([Fig. 2.2](#)). Formally, every feature is a function of strictly-prior history,

$$f = g(\{x_s : s < t\}), \quad (2.1)$$

a convention we will call the *as-of rule*. [Equation \(2.1\)](#) looks innocuous; by the end of the chapter you will recognise it as the single most effective leakage vaccine we possess.

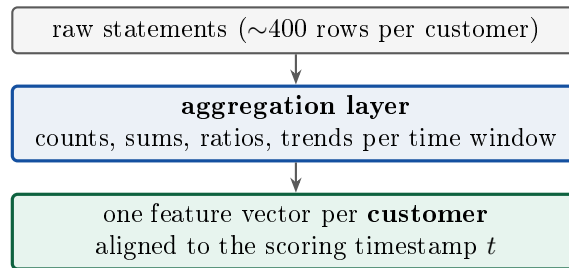


Figure 2.2: From transaction grain to prediction grain. The unit of prediction is the *customer at time t* , not the transaction; every aggregate is computed over a window that ends strictly before t ([Eq. \(2.1\)](#)).

2.2 Missing Data

Missing values are the most visible pathology and the most frequently mishandled. The mistake is not choosing the wrong imputer; it is imputing *before asking why the value is missing*. The “why” has a precise statistical formulation due to [Rubin \(1976\)](#), and the correct treatment depends on it.

2.2.1 Rubin’s Taxonomy of Missingness

Let $X = (X_{\text{obs}}, X_{\text{mis}})$ denote the complete data partitioned into its observed and missing parts, and let M be the missingness indicator matrix, $M_{ij} = 1$ if entry X_{ij} is unobserved. The joint behaviour of M and X defines three regimes.

Definition 2.1 (MCAR, Missing Completely At Random). The data are MCAR if missingness is independent of the data entirely:

$$P(M \mid X) = P(M).$$

Definition 2.2 (MAR, Missing At Random). The data are MAR if, conditional on what is observed, missingness carries no further information about the missing values:

$$P(M | X) = P(M | X_{\text{obs}}).$$

Definition 2.3 (MNAR, Missing Not At Random). The data are MNAR if missingness depends on the missing values themselves, even after conditioning on everything observed:

$$P(M | X) = P(M | X_{\text{obs}}, X_{\text{mis}}).$$

Example 2.1 (The three regimes in one dataset). In the mobile-money extract, all three regimes coexist. Random API timeouts that drop transaction rows are MCAR: the lost rows are a uniform sample. GPS coordinates missing for feature-phone users are MAR: missingness is fully explained by the *observed* device-type column. And applicants with erratic incomes who *decline* to link their full statement are MNAR: the probability that the income field is missing depends on the (unobserved) income itself.

The distinction matters because it dictates what imputation can and cannot recover. Under MCAR, complete-case analysis is unbiased (merely inefficient). Under MAR, model-based imputation that conditions on the observed columns can recover the joint distribution. Under MNAR, *no* imputation scheme that uses only the observed data is unbiased, but something better than unbiasedness is often available: the missingness itself is *signal*.

Remark 2.1 (Missingness as a feature). In credit scoring, MNAR is the rule, not the exception, and the fact that a value is missing is frequently more predictive than any imputed replacement. An applicant who declines to link a statement is telling you something. The standing recommendation of this book: whenever MNAR is plausible, add the binary indicator $m_{ij} = \mathbb{1}[x_{ij} \text{ missing}]$ as a feature in its own right, regardless of how (or whether) you impute the value.

2.2.2 Diagnosis Before Treatment

Three cheap diagnostics should precede any imputation decision, and all three appear in Lab 2.

First, *quantify*: the per-column missing fraction, sorted. Second, *test for MNAR-like structure*: compare the target rate across the missing indicator. If the 90-day default rate is 4.1% when GPS is present and 9.7% when it is absent, the indicator has earned a place in the feature set, a two-line test that has settled more imputation debates than any theory. Third, *inspect co-missingness*: when a block of columns is always missing together, the cause is usually a failed upstream join or a pipeline change on a particular date, an engineering fault to fix at the source, not a statistical pattern to model.

```

1 import pandas as pd
2
3 df = pd.read_parquet("mm_statements.parquet")
4
5 # 1. How much is missing, per column?
6 miss = df.isna().mean().sort_values(ascending=False)
7 print(miss.head(10))
8
9 # 2. Does missingness correlate with the target? (MNAR screen)
10 df["gps_missing"] = df["gps_lat"].isna().astype(int)
11 print(df.groupby("gps_missing")["default_90d"].mean())
12 # 0      0.041    <- GPS present
13 # 1      0.097    <- GPS missing: keep the indicator as a feature!
14
15 # 3. Co-missingness structure (block patterns => pipeline faults)
16 import missingno as msno
17 msno.matrix(df)

```

```
18 msno.heatmap(df)
```

Listing 2.1: Diagnosing missingness before treating it.

A practical corollary: when a column is missing for the vast majority of rows (say, above 60%), imputation manufactures a column that is mostly synthetic. Keep the indicator, and either drop the value column or accept that trees will treat the imputed constant as one more split point.

2.2.3 Imputation I: Simple Statistics with Indicators

The workhorse strategy replaces each missing numeric entry with a location statistic of its column computed *on the training split only*:

$$\hat{x}_{ij} = \text{median}(x_{.j}^{\text{train}}),$$

with the median preferred over the mean because transaction amounts are heavy-tailed (Section 2.4) and a single whale deposit should not drag the fill value. Categorical columns take the most frequent level or an explicit sentinel such as "UNKNOWN", the sentinel is usually better, because it preserves the information that the value was absent rather than silently inflating the modal category.

```
1 from sklearn.impute import SimpleImputer
2
3 num_imputer = SimpleImputer(strategy="median",
4                             add_indicator=True) # appends m_ij columns
5 cat_imputer = SimpleImputer(strategy="constant",
6                             fill_value="UNKNOWN")
7
8 X_train_num = num_imputer.fit_transform(X_train[NUM_COLS]) # fit on TRAIN
9 X_test_num  = num_imputer.transform(X_test[NUM_COLS])    # transform only
```

Listing 2.2: Simple imputation with missing indicators. The `fit` call learns statistics from the training data only; the same fitted object then transforms validation and test splits.

The phrase *training split only* is not a stylistic preference. Computing the median over all rows, train and test together, imports information about the test distribution into the training data. It is our first concrete instance of preprocessing leakage, and Section 2.7 will name it as cause number three on Huyen's list.

2.2.4 Imputation II: Model-Based Methods

When the missingness is MAR and the columns are correlated, the observed columns predict the missing ones, and two model-based schemes exploit this.

KNN imputation fills x_{ij} with the (distance-weighted) mean of the value in the k nearest complete rows, nearness being measured in the space of the remaining features. It respects local structure, a customer whose deposit pattern resembles other salaried customers receives a salaried-looking fill, at the price of an $O(n^2)$ neighbour search.

Iterative imputation, the `scikit-learn` realisation of the MICE family (Multiple Imputation by Chained Equations), models each incomplete column as a function of all the others and cycles until the fills stabilise:

$$x_{.j}^{(t)} \sim f_j(x_{.,-j}^{(t-1)}), \quad j = 1, \dots, d, \quad (2.2)$$

where f_j is a regressor (Bayesian ridge by default) fitted on the currently completed data with column j as target. With `sample_posterior=True`, the fills are draws rather than point estimates, which propagates imputation uncertainty in the spirit of proper multiple imputation.

```

1 from sklearn.experimental import enable_iterative_imputer # noqa
2 from sklearn.impute import KNNImputer, IterativeImputer
3
4 knn = KNNImputer(n_neighbors=5, weights="distance")
5
6 mice = IterativeImputer(max_iter=10, random_state=42,
7                          sample_posterior=True) # ~ MICE
8
9 X_train_i = mice.fit_transform(X_train[NUM_COLS])
10 X_test_i = mice.transform(X_test[NUM_COLS])

```

Listing 2.3: Model-based imputers. Both learn from the training split and transform all splits.

Both methods scale poorly to tens of millions of rows, and in production the cost-benefit calculus usually lands on simple-plus-indicator unless model-based imputation demonstrably lifts the validation metric. There is also a serving-time constraint that offline experiments hide: whatever imputer you fit must run *inside the prediction service*, on one row, in milliseconds. An iterative imputer that cycles ten regressors is a very different deployment artefact from a dictionary of medians.

2.2.5 A Decision Framework

Table 2.1 condenses the discussion into the decision table used in Lab 2. One row deserves emphasis: modern gradient-boosted tree libraries (??) handle missing values *natively* by learning a default branch direction per split (Chen and Guestrin, 2016), so for those models the honest answer to “which imputer?” is often “none, but keep the indicator anyway if MNAR is plausible.”

Table 2.1: Choosing an imputation strategy.

Situation	Recommended	Rationale
MCAR, < 5% missing	median / mode	bias negligible; cheapest
MAR, moderate	iterative (MICE) or KNN	exploits correlations with observed features
MNAR (credit!)	simple + missing indicator	the fact of missingness carries signal
> 60% missing	indicator only, or drop	imputed values would dominate the column
Tree ensembles	often <i>none</i>	trees route NaN via learned default directions

Imputing with statistics from the full dataset

A pipeline that calls `imputer.fit(X)` on the concatenated data before splitting has already leaked: every training row now carries a fill value computed partly from test rows. The damage is usually a fraction of an AUC point, small enough to survive review, large enough to flatter every offline benchmark and to evaporate on deployment. The structural fix is not discipline but architecture: put the imputer *inside* a **Pipeline** so that cross-validation re-fits it on each fold’s training portion automatically (Listing 2.12).

2.3 Outliers and Duplicates

2.3.1 Detecting Outliers

Three detectors cover the practical spectrum, in increasing order of sophistication.

The **z-score** flags points far from the mean in standard-deviation units, $|z_i| = |x_i - \mu|/\sigma > 3$. It presumes near-Gaussian data, a poor description of transaction amounts, and both μ and σ are themselves distorted by the outliers being hunted.

The **IQR rule** is the robust default: with Q_1, Q_3 the quartiles and $\text{IQR} = Q_3 - Q_1$, flag any point outside

$$[Q_1 - 1.5\text{IQR}, Q_3 + 1.5\text{IQR}]. \quad (2.3)$$

Quartiles are insensitive to the tail, so the fence does not move when a whale swims past.

The **Isolation Forest** (Liu et al., 2008) generalises to the multivariate case. It grows an ensemble of random trees, each isolating points by recursive random axis-aligned splits; anomalous points sit in sparse regions and are isolated in few splits. Averaging the isolation depth $h(x)$ over trees and normalising by the expected depth $c(n)$ of a binary search tree on n points yields the anomaly score

$$s(x) = 2^{-\mathbb{E}[h(x)]/c(n)} \in (0, 1], \quad (2.4)$$

with scores near 1 indicating anomalies. Its virtue for our case study is that it sees *combinations*: a KES 50,000 withdrawal is unremarkable, and a 3 a.m. transaction is unremarkable, but a KES 50,000 withdrawal at 3 a.m. from a dormant account is precisely the kind of joint outlier that univariate fences miss.

```

1 import numpy as np
2 from sklearn.ensemble import IsolationForest
3
4 amt = df["amount"]
5 q1, q3 = amt.quantile([.25, .75])
6 iqr = q3 - q1
7 iqr_mask = (amt < q1 - 1.5 * iqr) | (amt > q3 + 1.5 * iqr)
8
9 iso = IsolationForest(contamination=0.01, random_state=42)
10 df["anomaly"] = iso.fit_predict(df[["amount", "balance", "hour"]])
11 # -1 = anomaly, +1 = inlier

```

Listing 2.4: Univariate (IQR) and multivariate (Isolation Forest) outlier detection.

2.3.2 Treating Outliers, and When Not To

Detection is diagnosis; treatment is a modelling decision, and the options form a ladder of decreasing aggression. *Correct* entries that are provably wrong: the KES 9,999,999 agent test transaction has a known cause and a known fix (delete it, and file a ticket upstream). *Winsorize*, clip to percentiles,

$$\tilde{x}_i = \min(\max(x_i, P_1), P_{99}), \quad (2.5)$$

which folds the extreme tail into the last legitimate bin instead of discarding rows. *Transform*: heavy right tails frequently vanish under $\log(1+x)$, making explicit clipping unnecessary (Section 2.4). Or simply *scale robustly* and let the model cope: the robust scaler of Section 2.4 bounds the leverage of extreme points without touching them.

Deleting the signal

In fraud detection and credit risk, outliers are frequently *the phenomenon you are paid to detect*. The applicant whose account shows three enormous round-figure deposits the week before applying is not a data-quality problem, that pattern (briefly parking borrowed

money to inflate a balance) is a known gaming strategy. A cleaning script that winsorizes it away has laundered the evidence. The operational rule: **correct provable errors; clip or transform mechanical extremes; never delete behavioural extremes without a domain conversation.** Keep the raw column alongside any treated version until the model comparison settles which one earns its keep.

2.3.3 Duplicates and Entity Resolution

Exact duplicate rows arise from client retries, log merges, and self-inflicted oversampling; a single call to `drop_duplicates` dispatches them. The hard problem is *entity* duplication: the same real-world customer represented by several records that do not match on any key. In East African mobile-money data this is endemic, multiple SIM cards per person is normal consumer behaviour, and names transliterate inconsistently: "Wanjiku" versus "Wanjiko", or order-swapped surnames.

Definition 2.4 (Entity resolution). Entity resolution is the task of partitioning a set of records into groups such that all records in a group refer to the same real-world entity. The partition induces a canonical identifier (here, `customer_id`) used as the unit of aggregation and, crucially, of splitting.

```

1 # Exact row duplicates
2 df = df.drop_duplicates(subset=["txn_id"], keep="first")
3
4 # Entity resolution sketch: fuzzy name + exact DOB
5 from rapidfuzz import fuzz
6
7 def same_person(a, b):
8     name_sim = fuzz.token_sort_ratio(a["name"], b["name"])
9     return name_sim > 92 and a["dob"] == b["dob"]
10
11 # Group SIMs into one customer_id, aggregate features per CUSTOMER,
12 # and split by customer_id -- never by SIM.
```

Listing 2.5: Exact deduplication and a fuzzy entity-resolution sketch. In production this is a pipeline stage with human review of borderline matches, not a one-liner.

Why does this belong in a chapter that ends with leakage? Because failing to resolve entities *before splitting* scatters copies of the same customer across train and test. The model then “predicts” the test customer by recognising them from training, an optimistic bias that has nothing to do with generalisation. Deduplicate first; split by the resolved `customer_id`; and if you oversample a minority class, do it *after* splitting, or the duplicates you manufacture will cross the boundary themselves.

2.4 Scaling and Transformations

2.4.1 Why Scale, and With What

Gradient-based learners (logistic regression, SVMs, neural networks) and distance-based learners (k -NN, K-Means) are sensitive to feature magnitudes: a feature measured in shillings (10^0 – 10^6) dominates one measured as a ratio (0–1) both in the loss surface’s conditioning and in any Euclidean distance. Tree-based models split on order statistics and are indifferent to monotone rescaling, one more reason for their popularity on tabular data, but we do not build pipelines for one model family.

Three scalers cover practice. **Standardization** maps to zero mean and unit variance,

$$x' = \frac{x - \mu_{\text{train}}}{\sigma_{\text{train}}}; \quad (2.6)$$

min–max scaling maps to a bounded interval,

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}} \in [0, 1], \quad (2.7)$$

but is fragile because a single outlier redefines $x_{\max}$ and crushes everything else toward zero; and the **robust scaler**,

$$x' = \frac{x - \text{median}}{\text{IQR}}, \quad (2.8)$$

is the sensible default for transaction amounts, for the same reason the IQR fence was: quartiles ignore the tail.

Every fitted quantity in Eqs. (2.6) to (2.8), μ , σ , $x_{\min}$, $x_{\max}$, the median, the IQR, is a *parameter learned from data*, and the by-now-familiar rule applies: estimate on the training split, apply everywhere. Fitting a scaler on the full dataset before splitting is leakage cause number two (Section 2.7.2).

2.4.2 Taming Skew: Log and Power Transforms

Transaction amounts, incomes, and balances are heavy-tailed: most values are small, a few are enormous, and the histogram hugs the axis. Linear models struggle because a handful of points dominate the fit; distance-based methods struggle because the tail stretches the metric. The remedy is a monotone transformation that compresses the tail.

The quick fix is $\log(1 + x)$ for non-negative data. The principled family is the **Box–Cox transform** (Box and Cox, 1964), defined for $x > 0$ by

$$x^{(\lambda)} = \begin{cases} \frac{x^\lambda - 1}{\lambda} & \lambda \neq 0, \\ \ln x & \lambda = 0, \end{cases} \quad (2.9)$$

with λ chosen by maximum likelihood *on the training split* to make the result as Gaussian as possible. Because net cash-flow features can be negative, its extension, the **Yeo–Johnson transform** (Yeo and Johnson, 2000), handles all of $\mathbb{R}$ by applying a Box–Cox-like map to the positive part and a mirrored one to the negative part; it is the default in `scikit-learn`’s `PowerTransformer`. Strongest of all is the **quantile transform**, which maps each value through the empirical CDF onto a uniform or normal distribution, it flattens any shape, at a cost worth pausing on in a regulated industry: a feature defined as “the normal quantile of the empirical CDF of deposit volume” is hard to explain to a credit officer, let alone to a regulator reviewing an adverse-action decision.

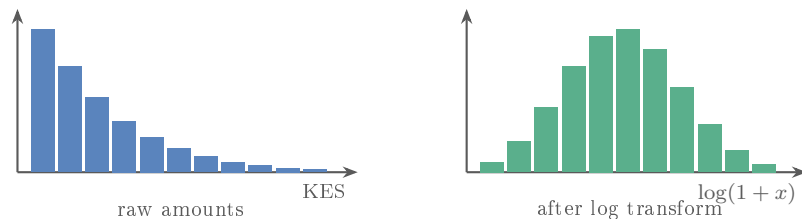


Figure 2.3: A heavy right tail (left) compressed toward symmetry by a log transform (right). Power transforms (Eq. (2.9)) generalise this idea with a data-driven exponent λ .

```

1 from sklearn.preprocessing import (PowerTransformer,
2                                   QuantileTransformer, RobustScaler)
3
4 pt = PowerTransformer(method="yeo-johnson", standardize=True)
5 qt = QuantileTransformer(output_distribution="normal",
6                           n_quantiles=1000)
7
8 X_train_t = pt.fit_transform(X_train) # lambda estimated on TRAIN
9 X_test_t  = pt.transform(X_test)     # applied, never re-estimated

```

Listing 2.6: Power and quantile transforms. Note the asymmetry between `fit_transform` on train and `transform` on test.

2.5 Encoding Categorical Variables

Most learning algorithms consume numbers, and most operational data is categorical: transaction type, county, device model, agent identifier. The encoding decision splits on one variable, *cardinality*, the number of distinct levels K , and on whether the levels carry a genuine order.

2.5.1 Low Cardinality: One-Hot and Ordinal

For $K \lesssim 15$ nominal levels, **one-hot encoding** maps level k to the indicator vector $e_k \in \{0, 1\}^K$. Two mechanical cautions. The K columns sum to one, so with an intercept they are perfectly collinear (the “dummy trap”); regularised and tree models shrug, but an unregularised linear model needs `drop="first"`. And at serving time a category the encoder never saw, a newly onboarded transaction type, must be handled by configuration, not by a 2 a.m. exception: setting `handle_unknown` to `"infrequent_if_exist"` folds rare and unseen levels into a shared bucket.

Ordinal encoding assigns consecutive integers and is correct exactly when the order is real: loan tiers bronze < silver < gold, education levels, delinquency buckets. Applying it to nominal data (counties!) invents a spurious geometry, Mombasa is not “three more than” Kisumu, that linear and distance-based models will happily exploit.

2.5.2 High Cardinality I: Frequency Encoding and the Hashing Trick

An agent-identifier column with $K \approx 10^4$ levels makes one-hot encoding a sparse liability: 10^4 columns, most seen a handful of times. Two cheap alternatives preserve a fixed footprint.

Frequency encoding replaces level c by its training-split relative frequency $\hat{p}(c)$: one dense column, monotone in popularity, capturing the often-predictive distinction between high-traffic and boutique agents. Unseen levels at serving time receive frequency 0, a sensible “never seen before” semantics.

The **hashing trick** dispenses with a fitted vocabulary altogether: a hash function maps any level, including levels that did not exist at training time, into a fixed number of buckets,

$$\phi(c) = h(c) \bmod d, \quad \phi: \mathcal{C} \rightarrow \{0, \dots, d-1\}. \quad (2.10)$$

Collisions (distinct levels sharing a bucket) inject noise, but the degradation is graceful and controllable through d , and the method needs no state beyond the hash function itself, which is why it is standard in large-scale advertising and fintech systems where the category universe churns daily.

```

1 # Frequency encoding: statistics from the train split only
2 freq = X_train["agent_id"].value_counts(normalize=True)
3 X_train["agent_freq"] = X_train["agent_id"].map(freq)
4 X_test["agent_freq"]  = X_test["agent_id"].map(freq).fillna(0)

```

```

5
6 # Hashing trick: no fitted vocabulary, unseen levels welcome
7 from sklearn.feature_extraction import FeatureHasher
8 h = FeatureHasher(n_features=256, input_type="string")
9 H = h.transform(df["agent_id"].astype(str))

```

Listing 2.7: Frequency encoding (fitted on train) and the stateless hashing trick.

2.5.3 High Cardinality II: Target Encoding

The most powerful classical encoder is also the most dangerous. **Target encoding** (Micci-Barreca, 2001) replaces level c by a *smoothed* estimate of the target mean among its rows:

$$\text{TE}(c) = \frac{n_c \bar{y}_c + m \bar{y}}{n_c + m}, \quad (2.11)$$

where n_c is the count of level c , $\bar{y}_c$ its empirical target mean, $\bar{y}$ the global mean, and m a smoothing constant. The estimator is a convex combination that shrinks rare levels toward the global prior: as $n_c \rightarrow \infty$ the level's own evidence dominates, while a level seen twice is pulled almost entirely to $\bar{y}$.

Remark 2.2 (Target encoding as empirical Bayes). Equation (2.11) is exactly the posterior mean of a Beta–Binomial model with prior strength m centred at $\bar{y}$: target encoding is shrinkage estimation wearing a feature-engineering hat. This view also explains its failure mode, with m too small, rare levels memorise their own labels.

Therein lies the danger. Each row's encoded value is computed from labels that *include its own*: the feature contains a whisper of the answer. On a level seen once, the whisper is a shout, $\text{TE}(c)$ with small m approaches y_i itself, and validation scores soar while production scores do not. This *self-leakage* is neutralised by **out-of-fold encoding**: partition the training data into folds; encode each fold using statistics fitted on the *other* folds; fit a final encoder on all training data for use on test and in serving.

```

1 from category_encoders import TargetEncoder
2 from sklearn.model_selection import KFold
3
4 X_enc = X_train.copy()
5 te = TargetEncoder(cols=["agent_id"], smoothing=20)
6
7 for tr, va in KFold(5, shuffle=True, random_state=42).split(X_train):
8     te.fit(X_train.iloc[tr], y_train.iloc[tr])
9     X_enc.iloc[va] = te.transform(X_train.iloc[va])
10
11 te_final = TargetEncoder(cols=["agent_id"], smoothing=20)
12 te_final.fit(X_train, y_train) # for test + serving
13 X_test_enc = te_final.transform(X_test)

```

Listing 2.8: Out-of-fold target encoding. Each training row is encoded by statistics that exclude its own label.

Looking ahead: the modern successor to target encoding is the *learned embedding*, a dense vector per level trained jointly with the model, which we will meet with neural networks in ?? and again, in its most spectacular form, with language models in ??.

2.6 Feature Engineering

2.6.1 Engineered versus Learned Features

“Doesn’t deep learning make feature engineering obsolete?” The promise is real for perceptual data: convolutional networks learn edge, texture, and object detectors (??); transformers learn token representations (??). But the claim quietly assumes the raw input *contains* the signal in a locally-structured form a network can mine. A flat transaction table does not: no architecture will spontaneously invent “ratio of outflows to inflows over the 90 days preceding the application” from rows it sees one at a time. For tabular, relational, operational data — still the substrate of most production ML (Huyen, 2022), feature engineering is where the domain knowledge enters the system, and this section is organised around the feature families that recur in behavioural credit scoring.

2.6.2 Domain Features for Mobile-Money Credit Scoring

Table 2.2 lists the five families that form the backbone of Lab 2’s feature set. Two disciplines govern all of them. First, the *as-of rule* of Eq. (2.1): every aggregate is computed over a window ending strictly before the scoring timestamp t , typically several windows (7, 30, 90 days) so the model can see both level and trend. Second, *lineage*: for each feature, record which upstream fields feed it and when they are written, a bookkeeping habit that will pay for itself in Section 2.7.

Table 2.2: Domain feature families for thin-file credit scoring.

Family	Examples	Why predictive
RFM	recency of last deposit; frequency of P2P transfers in 30/90 d; monetary volume in/out	the classic behavioural-credit triad: recent, regular, substantial activity signals capacity
Ratios	outflows/inflows; airtime share of spend; savings rate	scale-free, hence comparable across income levels
Velocity / trend	slope of balance over 90 d; growth in transaction count	distinguishes rising from collapsing cash-flow at equal averages
Stability	std. of weekly inflows; # distinct counterparties; agent diversity	income regularity for informal earners, the thin-file analogue of “salaried”
Behavioural flags	night-time transaction share; betting-paybill share; loan-app cascade	risk-culture signals; require fairness scrutiny (see below and ??)

Remark 2.3 (Behavioural flags and proxy discrimination). The last family deserves a caution flag of its own. A feature such as “share of spend at betting paybills” may be genuinely predictive and still function as a *proxy* for age, gender, or neighbourhood, attributes a lender may not lawfully score on. Chapter 1 introduced the fairness criteria; here the practical rule is procedural: every behavioural flag added to the feature set must be reviewed against the protected attributes it may proxy, documented in the feature dictionary, and revisited when Kenya’s Data Protection Act guidance evolves. Predictive power is a necessary, not a sufficient, condition for inclusion.

2.6.3 Datetime Features and Cyclical Encoding

Timestamps are feature goldmines, hour of day, day of week, day of month, distance to payday, but their periodic components need care. Encoded as an integer, hour 23 and hour 0 sit 23 units apart; on the clock they are neighbours. The fix embeds the periodic variable on the unit circle:

$$x_{\sin} = \sin \frac{2\pi t}{T}, \quad x_{\cos} = \cos \frac{2\pi t}{T}, \quad (2.12)$$

with period $T = 24$ for hours, 7 for weekdays, 12 for months. Both coordinates are needed: either alone maps two different times to the same value (Fig. 2.4). For the case study, two non-periodic datetime features matter as much: *distance to the nearest payday* (end-of-month salary cycles dominate Kenyan cash-flow) and *days since account opening* (tenure).

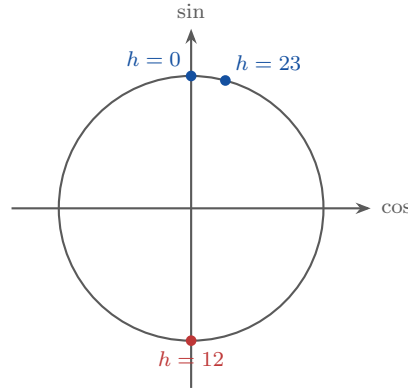


Figure 2.4: Cyclical encoding places periodic features on the unit circle: hours 23 and 0 become neighbours, as they are in reality, while hour 12 is maximally distant.

```

1 import numpy as np
2
3 df["hr_sin"] = np.sin(2 * np.pi * df["hour"] / 24)
4 df["hr_cos"] = np.cos(2 * np.pi * df["hour"] / 24)
5 df["dow_sin"] = np.sin(2 * np.pi * df["dayofweek"] / 7)
6 df["dow_cos"] = np.cos(2 * np.pi * df["dayofweek"] / 7)
7
8 # distance (in days) to the nearest month-end payday
9 dom = df["date"].dt.day
10 df["payday_dist"] = np.minimum(np.abs(dom - 1), np.abs(dom - 28))

```

Listing 2.9: Cyclical and calendar features.

2.6.4 Interactions, Binning, and Crosses

Linear models are additive; reality is not. Three constructions let simple models express non-additive structure, and let complex models be replaced by simple, auditable ones.

Interaction features multiply or divide raw features: $x_{\text{inflow}} \times \mathbb{I}[\text{salaried}]$ lets the effect of inflow volume differ by employment type. **Binning** (discretization) maps a continuous feature into quantile buckets; it is robust to outliers, captures non-monotone effects, and, decisively in lending, is the representation regulators know, because it is the native format of the credit *scorecard*. In scorecard practice each bin of each feature receives a *weight of evidence*,

$$\text{WoE}(\text{bin}) = \ln \frac{P(\text{bin} \mid y = 0)}{P(\text{bin} \mid y = 1)}, \quad (2.13)$$

the log-ratio of the bin's share among good versus bad outcomes; a logistic regression on WoE-coded bins is precisely the traditional scorecard (Siddiqi, 2017), and remains the deployment format many East African lenders' risk committees expect. **Feature crosses** concatenate categoricals (`county` $\times$ `device_type`) into a single high-cardinality feature, then encode it with the hashing trick, a cheap way to expose combinatorial structure.

```

1 from sklearn.preprocessing import KBinsDiscretizer, PolynomialFeatures
2
3 kb = KBinsDiscretizer(n_bins=10, encode="ordinal", strategy="quantile")

```

```

4
5 poly = PolynomialFeatures(degree=2, interaction_only=True,
6                           include_bias=False)
7
8 df["cty_dev"] = df["county"] + "_" + df["device_type"] # cross

```

Listing 2.10: Binning, polynomial interactions, and a manual categorical cross.

Remark 2.4. Tree ensembles discover interactions automatically along their split paths, which is why hand-crafted interactions matter most for linear and regularised models, and for the interpretable scorecards that regulation sometimes mandates. Engineering effort should follow the deployment constraint, not the other way round.

2.6.5 What Makes a Feature Good?

With the construction toolbox open, the curation question follows: which features earn a place? Four lenses, in the order you should apply them.

Importance. Measure each feature’s contribution, split gain for trees, SHAP values (Lundberg and Lee, 2017) model-agnostically. The recurring empirical pattern is sharp concentration: a minority of features carries most of the lift, and the tail is candidates for removal (unused features are not free, they are silent schema dependencies that will break a pipeline the day an upstream table changes).

Coverage. The fraction of rows where the feature is present. Low coverage is not disqualifying, under MNAR the indicator may be the value (Remark 2.1), but it bounds how much the imputed column itself can contribute.

Distribution overlap. The feature’s support in training must overlap its support in serving. The canonical counterexample (Huyen, 2022): a delivery-time model trained on Monday–Saturday data with a `DAY_OF_WEEK` feature and deployed on Sunday, coverage 100%, overlap zero, and the feature actively misleads.

Generality versus specificity. `IS_RUSH_HOUR` generalises better than `HOUR_OF_DAY`; it also discards information. There is no free choice here, only a trade-off to be settled by validation, and often by keeping both.

One diagnostic cuts across all four lenses and points forward to the final section: a feature whose association with the target is *suspiciously strong*, an AUC lift no domain story explains, is not a triumph. It is a leakage investigation waiting to be opened.

2.7 Data Leakage

2.7.1 Definition and Why It Is Dangerous

Definition 2.5 (Data leakage). Data leakage is the use, during training or model selection, of information that will not be available (or not available in the same form) at prediction time, most commonly, information about the label finding its way into the features or into the preprocessing statistics. Formally, leakage exists when the training-time feature map ϕ_{train} and the serving-time feature map ϕ_{serve} differ in distribution: the model is evaluated on $\phi_{\text{train}}(x)$ but will be applied to $\phi_{\text{serve}}(x)$ (Kaufman et al., 2012).

Leakage is dangerous for a reason that bears spelling out: *it makes every offline safeguard lie in the same direction*. Cross-validation, held-out testing, model comparison, all of them reward the leak, because the leaked information genuinely helps on the contaminated data. The failure surfaces only in production, after the loans have been disbursed, and its post-mortem is expensive precisely because the offline evidence looked impeccable. Two cautionary tales have become canon. A hospital’s CT-scan cancer model achieved excellent test performance, by

learning *which scanner* produced the image, because clinicians routed suspected-cancer patients to the advanced machine; at a second hospital with random machine assignment, the model collapsed (Huyen, 2022). And in the 2020 Ion Switching competition on Kaggle, the winning teams reverse-engineered test labels from an artefact of how the organisers had synthesised the data, leakage as a competitive sport.

2.7.2 Six Common Causes

Huyen’s field guide (Huyen, 2022) enumerates six mechanisms; between them they cover nearly every leakage post-mortem the authors have read or lived through.

1. **Splitting time-correlated data randomly instead of by time.** Future rows train the model that is evaluated on past rows. Nearly all operational data is time-correlated; split by time (Section 2.7.3).
2. **Scaling before splitting.** μ, σ, λ estimated on the full dataset import test-set statistics into training (Section 2.4).
3. **Imputing with full-data statistics.** The same mechanism through the imputer (Section 2.2).
4. **Duplicates spanning splits.** The same applicant, or the same applicant under a different SIM, appears on both sides of the boundary (Section 2.3.3). Deduplicate before splitting; oversample after.
5. **Group leakage.** Correlated examples divided across splits: two loans by one customer, two CT scans of one patient a week apart. Split by *group*.
6. **Leakage from the data-collection process.** The scanner story; in our case study, any field written by the collections team after delinquency. No statistical test reveals this class, only lineage and domain knowledge do.

Causes 1, 4, and 5 are *split-design* errors; causes 2 and 3 are *pipeline* errors with a structural fix (Section 2.7.5); cause 6 is an *epistemic* error that no code can catch for you.

2.7.3 Temporal and Group-Aware Splits

For the credit-scoring problem, the correct evaluation design mimics deployment: train on loans disbursed in one period, validate on the next, test on the one after (Fig. 2.5). Random K -fold on loans would let August repayment behaviour inform a February score, optimistic by construction, because deployment never enjoys that luxury.

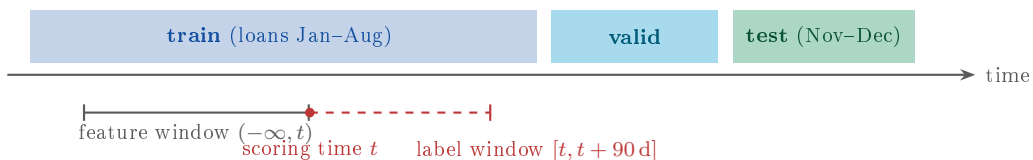


Figure 2.5: Temporal evaluation design for credit scoring. Each observation carries two clocks: features are computed over $(-\infty, t)$ and the label matures over $[t, t + 90 \text{ d}]$. Loans whose label window has not closed by a split boundary must be excluded, not assumed non-defaulting.

Two refinements complete the design. First, every observation carries *two clocks*: the feature window ends at the scoring time t , and the label matures over $[t, t + 90 \text{ d}]$. A loan disbursed 40 days before the boundary has an unresolved label; including it as “non-default” injects optimism exactly where the split should protect you. Exclude immature labels. Second, combine the

temporal split with *group awareness*: all loans of one (entity-resolved) customer stay on one side of each boundary, or cause 5 re-enters through the side door.

2.7.4 A Detection Protocol

Leakage is found by protocol, not luck. Four screens, ordered by cost.

1. **Correlation screen.** Rank features by $|\rho(f_j, y)|$ and interrogate the top of the list. Association that no domain story explains is guilty until proven innocent. Correlations can also hide in *pairs*, an employee's start date and end date are individually bland and jointly equal to tenure, so extend the screen to suspicious combinations.
2. **Ablation.** Remove a suspect feature and re-train. If the metric falls off a cliff, understand *why* the feature is that important before celebrating it.
3. **The too-good-to-be-true test.** Calibrate expectations against the literature and your baselines (??): an AUC of 0.99 for 90-day default from thin-file data is not a breakthrough; it is a bug report.
4. **Lineage review.** For every feature: which system writes the underlying field, when, and triggered by what? This is the only screen that catches cause 6.

```

1 import numpy as np
2
3 corr = (X_train
4         .apply(lambda c: np.corrcoef(c, y_train)[0, 1])
5         .abs().sort_values(ascending=False))
6 print(corr.head(5))
7 # days_since_collections_call    0.94    <-- !!
8 # outflow_inflow_ratio          0.31
9 # balance_slope_90d             0.27
10 # ...
11 # 'collections_call' is written AFTER delinquency:
12 # a post-outcome feature. Remove it; audit the lineage of its neighbours.
```

Listing 2.11: A two-minute correlation screen. The top-ranked feature is a post-outcome field, statistical smoke leading to a lineage fire.

Treating the test split as a sandbox

Every look at the test split spends it. Using test data to shortlist features, to pick hyper-parameters, or to decide when to stop iterating turns the final evaluation into an in-sample estimate, leakage through the analyst rather than through the pipeline. House discipline: the test split is radioactive material, touched exactly once, for the final report, after all decisions are frozen. Model selection lives on the validation split and on cross-validation within the training data.

2.7.5 The Structural Antidote: Pipelines

Causes 2 and 3, and target-encoding self-leakage, share a shape: a preprocessing step *fitted* on data it should not see. The cure is not vigilance but architecture. A `scikit-learn Pipeline` binds every fitted transformation to the estimator, so that any cross-validation or grid search re-fits the *entire* chain, imputers, scalers, encoders — on each fold's training portion only. Preprocessing leakage becomes not merely avoided but *structurally impossible*, and the very same object serialises for deployment, closing the training-serving gap (Section 2.8).

```

1 from sklearn.pipeline import Pipeline
2 from sklearn.compose import ColumnTransformer
3 from sklearn.impute import SimpleImputer
4 from sklearn.preprocessing import (OneHotEncoder, RobustScaler,
5                                   PowerTransformer)
6 from sklearn.linear_model import LogisticRegression
7 from sklearn.model_selection import cross_val_score, GroupKFold
8
9 num = Pipeline([
10     ("impute", SimpleImputer(strategy="median", add_indicator=True)),
11     ("power", PowerTransformer(method="yeo-johnson")),
12     ("scale", RobustScaler()),
13 ])
14 cat = Pipeline([
15     ("impute", SimpleImputer(strategy="constant", fill_value="UNK")),
16     ("onehot", OneHotEncoder(handle_unknown="infrequent_if_exist",
17                               min_frequency=50)),
18 ])
19 prep = ColumnTransformer([("num", num, NUM_COLS),
20                           ("cat", cat, CAT_COLS)])
21 model = Pipeline([("prep", prep),
22                   ("clf", LogisticRegression(max_iter=2000))])
23
24 # Every fold re-fits imputers/scalers/encoders on that fold's
25 # training portion only:
26 scores = cross_val_score(model, X, y,
27                           cv=GroupKFold(n_splits=5),
28                           groups=X["customer_id"],
29                           scoring="roc_auc")

```

Listing 2.12: The leakage-safe pattern that every model in this book will reuse: preprocessing and estimator in one Pipeline, evaluated with a group-aware split.

Listing 2.12 is the chapter in one listing: median imputation with indicators (Section 2.2), a Yeo–Johnson transform and robust scaling (Section 2.4), sentinel-imputed one-hot encoding with unknown-category handling (Section 2.5), all fitted inside the cross-validation loop and split by resolved customer (Sections 2.3.3 to 2.7.3). For a fully deployment-faithful evaluation, replace `GroupKFold` with a time-ordered variant that respects Fig. 2.5; we return to time-aware cross-validation in Chapter 3.

2.8 From Pipelines to Feature Platforms

A closing bridge to Part III. Everything in this chapter was written as if one notebook computed the features once. In production, the same feature must be computed in *two places*: in batch, over history, to build training sets; and online, in milliseconds, when a live application arrives. Any divergence between the two implementations, a different null-handling branch, a window boundary off by one day, is *training-serving skew*: the deployed model receives features from a distribution it was never trained on, a self-inflicted cousin of leakage.

The industrial response is the *feature store*: a registry in which each feature is defined once, with its transformation logic, owner, and lineage, and served consistently to both training and inference paths. Its load-bearing guarantee is *point-in-time correctness*, the store answers “what was the value of `deposits_90d` for customer *c* as of time *t*?”, which the reader will recognise as Eq. (2.1) promoted from convention to infrastructure. When we build the course’s serving stack in ??, the feature dictionary you assemble in Lab 2 becomes the seed of exactly such a registry; and the drift monitoring of ?? watches the distribution-overlap lens of Section 2.6 continuously, in production, forever.

Lab 2: Messy Data & Leakage-Safe Features (1.5 h)

Dataset: a synthetic-but-realistic mobile-money statement dump (~180,000 transactions, ~4,000 customers) with injected pathologies: duplicated rows, three date formats, amounts stored as strings ("1,500/-"), MNAR-missing GPS, multi-SIM customers, and *two planted leaky columns*.

1. **Audit** (20 min). Profile missingness, duplicates, and validity-rule violations. Classify each incomplete column as MCAR / MAR / MNAR with evidence ([Listing 2.1](#)).
2. **Clean and aggregate** (30 min). Parse amounts and dates; resolve entities to a canonical `customer_id` ([Listing 2.5](#)); build customer-level RFM, ratio, and cyclical features aligned to a scoring timestamp, respecting the as-of rule ([Eq. \(2.1\)](#)).
3. **Hunt the leak** (20 min). Run the correlation screen ([Listing 2.11](#)) and a lineage argument to identify the two planted leaky features; justify their removal in writing.
4. **Pipeline** (20 min). Assemble the full `ColumnTransformer` pipeline ([Listing 2.12](#)); evaluate with group-aware cross-validation; report ROC–AUC *with* and *without* the leaky columns and comment on the gap.

Deliverable: the completed notebook, pushed to your course repository before Sunday 23:59 EAT, with a short markdown cell per step recording your reasoning.

Assignment 2 (graded, 20 marks)

Extend the Lab 2 notebook into a short technical report (maximum four pages of prose plus the notebook) addressed to the lender’s head of credit risk.

1. **Data-quality audit (5 marks)**. A table of the ten worst columns by quality dimension ([Fig. 2.1](#)), each with the evidence, the MCAR/MAR/MNAR classification where applicable, and the chosen remedy.
2. **Feature dictionary (5 marks)**. Fifteen or more engineered features across at least four families of [Table 2.2](#), each documented with: definition, window, upstream fields (lineage), write-time of each upstream field, and a one-line fairness note where relevant ([Remark 2.3](#)).
3. **Leakage memo (6 marks)**. Identify both planted leaks; explain the mechanism of each by reference to the six causes of [Section 2.7.2](#); quantify the offline AUC inflation each produces; and state the process change that would prevent its recurrence.
4. **Reproducible pipeline (4 marks)**. A single `Pipeline` object producing your final cross-validated ROC–AUC under a group-aware split, serialised with `jobjlib` and re-loadable by the marker.

Rubric. Full marks require evidence, not assertion: every classification, removal, and design choice must cite a diagnostic output or a lineage fact. Submissions whose pipeline fits any preprocessing step outside cross-validation forfeit the pipeline marks regardless of the score achieved, in this course, as in production, a leaky number is worth less than an honest one. Due: start of the Week 4 session.

Exercises

1. Give one example from a Kenyan health-informatics setting (e.g. clinic electronic records) of each missingness regime, MCAR, MAR, and MNAR, and state, for each, whether mean

imputation yields biased estimates of the population mean.

2. Prove that under MCAR (Definition 2.1), the complete cases are a random subsample of the data, and hence that complete-case estimates of the mean are unbiased.
3. The iterative imputer of Eq. (2.2) is run with `sample_posterior=False` (point estimates). Explain why the resulting completed dataset understates the variance of the imputed columns, and what consequence this has for downstream standard errors.
4. A junior analyst z-score-standardises transaction amounts and removes all rows with $|z| > 3$, then reports that default prediction AUC *dropped*. Offer two distinct explanations consistent with this chapter.
5. Show that min–max scaling (Eq. (2.7)) maps a dataset’s maximum outlier to exactly 1 regardless of its magnitude, and compute what happens to the scaled value of the median when the maximum is multiplied by 10.
6. Derive Eq. (2.11) as the posterior mean of a Beta–Binomial model: specify the prior in terms of m and $\bar{y}$, and verify the limits $n_c \rightarrow 0$ and $n_c \rightarrow \infty$.
7. Two features, `contract_start_date` and `contract_end_date`, each correlate weakly with employee churn, yet a model using both achieves implausibly high accuracy. Explain the mechanism, and propose a screen that would catch this class of pairwise leakage.
8. For the hashing trick (Eq. (2.10)) with K levels uniformly distributed over d buckets, derive the expected number of colliding *pairs* of levels, and evaluate it for $K = 10^4$, $d = 256$.
9. The lender’s fraud team proposes the feature “number of calls from the customer to the support line in the past 7 days.” Using the lineage lens of Section 2.7.2, describe a plausible mechanism by which this feature could leak the label, and a redesign that keeps its legitimate signal.
10. A colleague argues that because gradient-boosted trees handle missing values natively, Lab 2’s imputation work is wasted effort. Write a five-sentence rebuttal that references at least the MNAR indicator argument (Remark 2.1) and the training–serving consistency argument (Section 2.8).

Key Terms

as-of rule · Box–Cox / Yeo–Johnson transform · cyclical encoding · data leakage · entity resolution · feature cross · feature store · frequency encoding · group leakage · hashing trick · Isolation Forest · MCAR / MAR / MNAR · MICE · missing indicator · one-hot encoding · out-of-fold encoding · point-in-time correctness · robust scaling · target encoding · training–serving skew · weight of evidence · winsorizing

Further Reading

The pipeline-centred treatment of data preparation in Géron (2022, Ch. 2) complements this chapter with fully worked code; Huyen (2022, Ch. 4–5) is the definitive production-oriented treatment of feature engineering and data leakage, and the source of the six-cause taxonomy of Section 2.7.2. Rubin (1976) founded the missing-data framework of Section 2.2; Kaufman et al. (2012) formalise leakage with instructive competition case studies; Micci-Barreca (2001) introduced smoothed target encoding; and Siddiqi (2017) is the standard reference for the score-card and weight-of-evidence tradition that still frames regulatory expectations in lending. For

evidence that mobile-phone behavioural features predict repayment in African markets, see Bjorkegren and Grissen (2020).

Part II

Deep Learning and Generative AI

Part III

MLOps and Production Systems

Appendices

Capstone Project

Solutions to Exercises

This appendix collects worked solutions, one section per chapter. Solutions are deliberately fuller than a marking scheme requires: they model the style of argument — framing before formalism, business units before metrics, honesty caveats stated explicitly — that the assignments and the capstone are graded against.

Chapter 1: Introduction to ML and End-to-End ML Projects

Solution to Exercise 1 (Framing). (a) **Streaming fraud detection with a 300-alerts/day operations team.**

Mitchell specification.

- T : score each transaction at initiation, before settlement, and emit an alert for those most likely to be fraudulent.
- E : historical transaction logs with fraud labels arriving with a delay of days to weeks, from chargebacks, customer complaints, and investigator dispositions; labels are incomplete (undetected fraud is labelled “legitimate”).
- P : fraud value captured (or recall) *at a fixed budget of 300 alerts per day* — operationally, precision@ k with $k = 300$ /day, weighted by transaction value.

Framing block. **Objective:** minimize net fraud losses subject to the investigation-capacity constraint and a false-positive customer-friction budget. **ML task:** binary classification (or anomaly scoring) on a *stream*; the drift speed of adversarial behaviour argues for online or frequently retrained batch learning (Table 1.4), evaluated prequentially. **Metric:** value-weighted recall at the 300-alert budget; secondary, time-to-detect for new fraud patterns. **Baseline:** the incumbent rules engine — its capture rate at the same alert volume is the number to beat. **Constraints:** per-transaction scoring latency in the tens of milliseconds; adversarial drift; delayed and censored labels; rollback capability for bad online updates.

Two honesty points earn full marks: the label censoring (undetected fraud contaminates the negatives, so measured recall is optimistic), and the feedback loop — blocking transactions changes which frauds are ever observed, the streaming analogue of the selective-labels problem of Section 1.4.

(b) **Thirty-day readmission from clinical pathway data.**

Mitchell specification.

- T : at discharge, estimate the probability that the patient is readmitted within 30 days.

- *E*: historical admission episodes — diagnoses, procedures, length of stay, vitals trajectories, prior utilization — joined to the observed readmission outcome.
- *P*: recall (or expected cost captured) at the capacity of the post-discharge intervention programme (follow-up calls, home visits); PR-AUC as the threshold-free summary, given a low base rate.

Framing block. **Objective:** allocate a fixed post-discharge intervention capacity to the patients whose readmission it is most likely to prevent. **ML task:** binary classification at the discharge-decision point; batch retraining is adequate (clinical drift is slow relative to fraud). **Metric:** recall at the intervention budget; per-segment calibration, since resource allocation across wards and demographics will be scrutinized. **Baseline:** the incumbent clinical risk score (e.g. a LACE-style index) or current clinician triage. **Constraints:** explainability to clinicians; data-protection compliance for health records; fairness across demographic groups; label subtleties — death within 30 days is a competing risk that must be handled explicitly, not silently coded as “no readmission”.

The deepest framing observation: the deployment intent is *interventional* (we act on high-risk patients), so post-deployment outcome data is shaped by the model’s own decisions — future retraining must account for the intervention, exactly the hidden feedback loop of [Section 1.4](#).

Solution to Exercise 2 (ML or not).

(a) **Blocking transactions above a regulatory limit: rule.** The requirement *is* a rule — crisp, complete, auditable, and legally fixed. ML could only approximate it, adding error where zero error is mandated and destroying auditability. Right-hand column of [Table 1.1](#) on every criterion.

(b) **SIM-swap account takeover: ML, wrapped in rules.** The signal is high-dimensional and behavioural (device changes, dormancy-then-drain patterns, atypical beneficiaries), and the adversary adapts, so any static rulebook rots within months — the left-hand column on every criterion. The mature design keeps hard rules as guardrails (e.g. mandatory step-up authentication within N hours of a SIM change) with an ML risk score ranking the grey zone; the rules also generate the labelled outcomes that train the model, the “rules first, ML second” pattern of [Section 1.2.3](#).

(c) **Routing support tickets by language: ML, but off the shelf.** Hand-written rules (character ranges, keyword lists) are brittle across code-switching and short texts, so learning is justified; but language identification is a solved commodity problem, and a lightweight pretrained classifier meets the accuracy, latency, and cost bar. The build-or-not lesson: “use ML” does not imply “train your own” — the total cost-of-ownership argument of [Table 1.1](#) favours the smallest maintained artifact that clears the requirement.

Solution to Exercise 3 (ERM: mean and median as optimal constants).

Squared error. For a constant predictor c , the empirical risk is

$$J(c) = \frac{1}{n} \sum_{i=1}^n (y_i - c)^2, \quad J'(c) = -\frac{2}{n} \sum_{i=1}^n (y_i - c),$$

and setting $J'(c) = 0$ gives $c^* = \frac{1}{n} \sum_i y_i = \bar{y}$, the sample mean; $J''(c) = 2 > 0$ confirms the minimum.

Absolute error. $G(c) = \frac{1}{n} \sum_i |y_i - c|$ is convex and piecewise linear, so we argue by subgradient. For c not equal to any y_i ,

$$G'(c) = \frac{1}{n} \left(\#\{i : y_i < c\} - \#\{i : y_i > c\} \right),$$

which is negative while fewer than half the points lie below c and positive once more than half do. The minimum is therefore attained where the two counts balance: at the sample median

(for even n , anywhere on the interval between the two middle order statistics — the minimizer need not be unique).

Connection to Section 1.6.2. Minimizing RMSE trains the model toward the conditional *mean*, which is pulled by extreme outcomes; minimizing MAE trains it toward the conditional *median*, which is robust to them. Choosing between Eq. (1.4) is therefore choosing *which summary of the target distribution the model learns* — a business decision (how do errors actually cost?) dressed as a metric choice.

Solution to Exercise 4 (Cost-sensitive thresholds).

Two-way derivation. For an applicant with calibrated default probability $\hat{p}$, compare conditional expected costs:

$$\mathbb{E}[C \mid \text{approve}] = \hat{p} c_{\text{FN}}, \quad \mathbb{E}[C \mid \text{decline}] = (1 - \hat{p}) c_{\text{FP}}.$$

Decline exactly when $\hat{p} c_{\text{FN}} > (1 - \hat{p}) c_{\text{FP}}$; rearranging,

$$\hat{p} > \frac{c_{\text{FP}}}{c_{\text{FP}} + c_{\text{FN}}} = t^*,$$

which is Eq. (1.6). The derivation *uses* calibration: $\hat{p}$ must be interpretable as a probability for the expectations to be meaningful, which is why calibration is a precondition, not a nicety.

Three-way extension. Add the action *refer*, costing a review fee c_R and (idealizing) resulting in the correct decision, so it incurs no error cost. The three conditional expected costs are $\hat{p} c_{\text{FN}}$ (approve), $(1 - \hat{p}) c_{\text{FP}}$ (decline), and c_R (refer). Comparing pairwise:

- approve beats refer iff $\hat{p} c_{\text{FN}} < c_R$, i.e. $\hat{p} < t_L := c_R / c_{\text{FN}}$;
- decline beats refer iff $(1 - \hat{p}) c_{\text{FP}} < c_R$, i.e. $\hat{p} > t_U := 1 - c_R / c_{\text{FP}}$.

The optimal policy is therefore: *approve* if $\hat{p} < t_L$, *refer* if $t_L \leq \hat{p} \leq t_U$, *decline* if $\hat{p} > t_U$. The referral band is non-empty iff $t_L < t_U$, i.e.

$$c_R < \frac{c_{\text{FN}} c_{\text{FP}}}{c_{\text{FN}} + c_{\text{FP}}},$$

the harmonic combination of the two error costs; cheap review widens the band, expensive review collapses it to the single threshold t^* — which, one checks, always lies inside $[t_L, t_U]$ when the band exists. This is exactly the “ $\hat{p}$ near t^* routes to a human” edge of Fig. 1.9, now with the band derived rather than asserted. (A fuller model would let the human err too; the band then narrows in proportion to reviewer accuracy, a good extension discussion.)

Solution to Exercise 5 (Leakage audit).

(a) **Number of repayment reminders sent: target leakage.** Reminders exist only after origination and are triggered by delinquency — the feature is a downstream *consequence* of the label. At application time it is identically empty, so a model trained with it learns to read the outcome off its own paperwork and collapses in production (Definition 1.3).

(b) **Median mobile-money balance over the 90 days before application: safe — conditional** on discipline. The window $[t_0 - 90\text{d}, t_0]$ respects the as-of contract of Table 1.6. The residual risks to state for full marks: the feature must be computed from event data as it stood at t_0 (late-arriving corrections or restated ledgers re-introduce temporal leakage), and the training-time computation must share code with the serving-time computation or training/serving skew replaces leakage as the failure mode.

(c) **Standard deviation of balance over the loan term: temporal leakage.** The loan term lies strictly *after* the origination decision the model is meant to inform. The feature is computable in the historical warehouse (which is why it slips into training sets) but can never be computed at prediction time. Textbook violation of the as-of rule.

(d) **Merchant-ID one-hot with random row-wise splits: group leakage.** Repeat merchants appear on both sides of a random split, so the identifier lets the model memorize *who* defaulted rather than learn *which behaviours* predict default; validation scores inflate and the model is useless for the new applicants it exists to score (whose IDs it has never seen). Remedies: split by merchant group *and* time; drop the raw identifier in favour of behavioural aggregates with proper as-of semantics; if entity-level history is genuinely predictive, encode it as backdated features (e.g. prior loans, prior repayment record as of t_0), which are legitimate.

Solution to Exercise 6 (Bias–variance: the vanishing cross-terms).

Setup as in Section 1.9.1: $y = g(x) + \varepsilon$ with $\mathbb{E}[\varepsilon] = 0$, $\text{Var}(\varepsilon) = \sigma^2$, ε independent of the training sample S ; $\hat{f}$ is trained on S ; $\bar{f}(x) = \mathbb{E}_S[\hat{f}(x)]$. Fix x and expand (all expectations over S and ε jointly):

First cross-term. Expanding $\mathbb{E}[(g + \varepsilon - \hat{f})^2] = \mathbb{E}[(g - \hat{f})^2] + 2\mathbb{E}[\varepsilon(g - \hat{f})] + \mathbb{E}[\varepsilon^2]$, the middle term factorizes because ε is independent of S and $\hat{f}$ is a function of S (and $g(x)$ is a constant for fixed x):

$$\mathbb{E}[\varepsilon(g - \hat{f})] = \mathbb{E}[\varepsilon] \mathbb{E}_S[g - \hat{f}] = 0 \cdot \mathbb{E}_S[g - \hat{f}] = 0.$$

Independence is used to factorize; the zero mean of ε kills the product. This leaves $\mathbb{E}[(g - \hat{f})^2] + \sigma^2$.

Second cross-term. Insert and subtract $\bar{f}$: $(g - \hat{f}) = (g - \bar{f}) + (\bar{f} - \hat{f})$, so

$$\mathbb{E}_S[(g - \hat{f})^2] = (g - \bar{f})^2 + 2(g - \bar{f}) \mathbb{E}_S[\bar{f} - \hat{f}] + \mathbb{E}_S[(\bar{f} - \hat{f})^2].$$

Here $(g - \bar{f})$ is a deterministic quantity (both terms are fixed numbers once x is fixed), so it exits the expectation; and by the *definition* of $\bar{f}$, $\mathbb{E}_S[\bar{f} - \hat{f}] = \bar{f} - \mathbb{E}_S[\hat{f}] = 0$. The cross-term vanishes, leaving Bias² + Variance + σ^2 , which is Eq. (1.7). Summary of where each assumption bites: independence of ε and S kills the first cross-term; the definition of $\bar{f}$ as the S -average kills the second.

Solution to Exercise 7 (Splits, empirically).

A model answer on synthetic data with controlled drift (students' numbers on real data will differ in size, not in sign):

```

1 import numpy as np
2 import pandas as pd
3 from sklearn.linear_model import LogisticRegression
4 from sklearn.metrics import average_precision_score
5 from sklearn.model_selection import train_test_split
6 from sklearn.pipeline import Pipeline
7 from sklearn.preprocessing import StandardScaler
8
9 rng = np.random.default_rng(8401)
10 n, d = 30_000, 10
11 t = np.sort(rng.uniform(0, 1, n))          # "application date" in [0,1]
12 X = rng.normal(size=(n, d))
13 w = np.array([2.0, -1.5, 1.0, 0.5, 0, 0, 0, 0, 0, 0])
14 drift = 2.5 * t                             # concept drifts over time
15 logits = X @ w + drift * X[:, 3] - 3.2
16 y = rng.binomial(1, 1 / (1 + np.exp(-logits)))
17
18 model = Pipeline([("s", StandardScaler()),
19                  ("m", LogisticRegression(max_iter=2_000,
20                  class_weight="balanced"))])
21
22 # (a) Random shuffled split -- time ignored
23 Xa_tr, Xa_te, ya_tr, ya_te = train_test_split(
24     X, y, test_size=0.25, stratify=y, random_state=8401)
25 pa = model.fit(Xa_tr, ya_tr).predict_proba(Xa_te)[:, 1]
```

```

26
27 # (b) Temporal split -- train on the past, test on the future
28 cut = int(0.75 * n) # rows already time-ordered
29 pb = (model.fit(X[:cut], y[:cut])
30       .predict_proba(X[cut:][:, 1]))
31
32 print(f"random split PR-AUC: {average_precision_score(ya_te, pa):.3f}")
33 print(f"temporal split PR-AUC: {average_precision_score(y[cut:], pb):.3f}")

```

Listing B.1: Random vs. temporal split under drift: the random split reports an optimistic number that no deployment can honour.

Direction and explanation. The random split scores higher. Under shuffling, training and test rows are interleaved in time, so the training set contains examples from the *same regime* as every test example — the model has effectively seen the future, and the drift term never has to be extrapolated. The temporal split forces genuine forward prediction into a drifted regime, which is exactly what deployment does (Fig. 1.6). **The honest number is the temporal one;** the gap between the two is not noise but a first, free estimate of how much drift will cost in production — worth reporting in the decision memo of Section 1.10.

Solution to Exercise 8 (Fairness).

Let F_A, F_B be the score CDFs of the calibrated model $\hat{p}$ in the two regions, with base rates $\pi_A = \mathbb{E}[\hat{p} \mid A] = 0.03$ and $\pi_B = \mathbb{E}[\hat{p} \mid B] = 0.09$ (calibration plus the tower property force the group mean score to equal the group base rate). Approval is $\hat{p} \leq t$ for a single global t .

Impossibility across thresholds. Demographic parity at *every* threshold would mean $F_A(t) = F_B(t)$ for all t , i.e. identical score distributions — hence identical means, contradicting $0.03 \neq 0.09$. So there is no global-threshold policy family satisfying parity uniformly.

The single-threshold case. At one fixed t , parity requires the accident $F_A(t) = F_B(t)$ at that exact point. But any calibrated model that actually uses the risk information distinguishing the regions places more probability mass at higher scores for region B (in the natural case, F_B is stochastically dominated by — i.e. sits below — F_A), giving $F_B(t) < F_A(t)$: region B’s approval rate is strictly lower at every useful threshold. Even in contrived cases where the CDFs cross at some t , the coincidence is knife-edge — it fails for neighbouring thresholds and evaporates at the first retrain — so it cannot ground a policy. The structural conclusion: with unequal base rates, a calibrated score plus a single global threshold *cannot* deliver demographic parity; achieving parity would require group-specific thresholds, which raises its own legal and ethical questions, or abandoning calibration — the conflict promised in Section 1.11.

Proposal to the regulator. A defensible position for the running case: retain per-group *calibration* (an uncalibrated score corrupts every downstream decision through Eq. (1.6)) and adopt *equal opportunity* — equal true-positive rates among repayers — as the operative criterion, i.e. good borrowers in both regions face the same approval odds; then justify any residual approval-rate gap by the base-rate difference while committing to close it upstream (product design, data enrichment for the thin-file region). **Post-deployment monitoring:** per-region approval rate, TPR/FPR among matured labels, calibration curves, and their drift over time — the fairness axis of the Week-13 monitors, reported segment-by-segment as Section 1.11 prescribes.

Solution to Exercise 9 (Hybrid architecture).

A full-marks answer reproduces the topology of Fig. 1.9 with the component classification made explicit:

- **Classical ML (the decision core).** The gradient-boosted or logistic scoring model with its leakage-safe feature pipeline (Listing 1.5), served behind the versioned API of Listing 1.7. It alone emits $\hat{p}$; it is trained, calibrated, validated, and monitored under the discipline of Chapters 1–4 and 11–13.

- **LLM components (the language edges).** (i) A RAG service over lending policy, KYC regulation, and product documents, answering underwriter questions with citations; (ii) an extraction model converting unstructured inputs (statements, free-text loan purpose, chat) into *typed features* that enter the classical pipeline — validated like any other upstream data source; (iii) an agent orchestrating the workflow: gather data, call the scoring API, draft the reason-coded communication.
- **Human review.** Applications with $\hat{p} \in [t_L, t_U]$ — the referral band derived in Solution 4 — route to a credit officer, with the agent’s brief attached.

Where calibration must be preserved end-to-end — the marking crux: the probability $\hat{p}$ must flow *numerically unaltered* from the scorer through the API to the thresholding policy and into any human-facing brief. The agent may *explain* the score; it must never *re-estimate*, round, re-rank, or paraphrase it into a different number, and no LLM output may feed back into the decision path — the moment a stochastic text generator sits between the calibrated score and the threshold of Eq. (1.6), the optimality of the policy, the audit trail, and the fairness monitoring of Solution 8 are all void. Diagrammatically: every arrow into the approve/refer/decline node originates at the classical scorer; LLM arrows terminate at humans or at the scorer’s *inputs*, never at its output.

Chapter 2: Data Preparation and Feature Engineering

Solution to Exercise 2.1. Consider the electronic medical records of a network of Kenyan outpatient clinics.

MCAR. A batch of laboratory results is lost when the connection between the lab analyser and the records system fails for an afternoon. Which patients’ rows are affected is determined by the timing of the outage, not by the patients or their values: $P(M | X) = P(M)$ (Definition 2.1). Mean imputation of the missing lab values yields an *unbiased* estimate of the population mean (the missing rows are a random sample), although it understates the variance (Exercise 2.3).

MAR. Blood-pressure readings are missing far more often at rural dispensaries that lack a working monitor than at urban clinics. Conditional on the *observed* facility-type column, missingness carries no further information about the reading itself (Definition 2.2). Unconditional mean imputation is *biased* whenever blood pressure differs systematically by facility type (urban and rural patient populations differ): the imputed mean is dominated by urban readings. Imputation that conditions on facility type — a group-wise mean, or the model-based imputers of the chapter — removes the bias.

MNAR. HIV-test results are missing because some patients decline testing, and the propensity to decline is associated with the patient’s own (unobserved) status and risk perception: $P(M | X)$ depends on X_{mis} (Definition 2.3). Mean imputation is *biased*, and no imputation scheme built from the observed data alone removes the bias; per Remark 2.1, the honest treatment records the refusal itself as an indicator feature and interprets downstream estimates with the selection mechanism in mind.

In summary, mean imputation is unbiased for the mean only under MCAR; under MAR it requires conditioning on the drivers of missingness; under MNAR it is biased regardless.

Solution to Exercise 2.2. Let $R_i \in \{0, 1\}$ indicate that row i is a complete case, and let X_i be its (complete-data) value. MCAR states $P(R | X) = P(R)$, i.e. $R \perp X$. The density of X among complete cases is, by Bayes’ rule,

$$f(x | R = 1) = \frac{P(R = 1 | X = x) f(x)}{P(R = 1)} = \frac{P(R = 1) f(x)}{P(R = 1)} = f(x),$$

where the middle equality uses independence. The complete cases are therefore distributed identically to the full population — a random subsample. Consequently, with $n_{cc} = \sum_i R_i$ the (random) number of complete cases,

$$\mathbb{E} \left[\frac{1}{n_{cc}} \sum_{i: R_i=1} X_i \middle| n_{cc} > 0 \right] = \mathbb{E}[X],$$

by taking the expectation first conditionally on the pattern $(R_1, \dots, R_n)$ — under which each retained X_i has mean $\mathbb{E}[X]$ by independence — and then over the pattern. The complete-case mean is unbiased; the only cost of MCAR missingness is the loss of efficiency from the reduced sample size. $\square$

Solution to Exercise 2.3. With point-estimate imputation (`sample_posterior=False`), every imputed entry is set to the conditional-mean prediction $\hat{\mathbb{E}}[X_{\text{mis}} | X_{\text{obs}}]$ of the final regression sweep of Eq. (2.2). Decompose the true variance by the law of total variance:

$$\text{Var}(X) = \underbrace{\mathbb{E}[\text{Var}(X | X_{\text{obs}})]}_{\text{residual spread}} + \underbrace{\text{Var}(\mathbb{E}[X | X_{\text{obs}}])}_{\text{explained spread}}.$$

The imputed values realise only the second term: they sit exactly on the conditional-mean surface, with zero residual spread. The completed column’s sample variance therefore converges to $\text{Var}(\mathbb{E}[X | X_{\text{obs}}])$, understating $\text{Var}(X)$ by the entire residual component $\mathbb{E}[\text{Var}(X | X_{\text{obs}})]$, and the understatement grows with the missing fraction.

Downstream, any procedure that treats the completed data as if observed — ordinary least squares, a t -test, a confidence interval for a feature mean — inherits standard errors that are too small: intervals are too narrow and p -values anti-conservative, because the analysis pretends the imputed entries carry the same information as real measurements. The remedies are those of proper multiple imputation: draw fills from the posterior (`sample_posterior=True`), create M completed datasets, and combine estimates with Rubin’s rules, which add the between-imputation variance the point-estimate shortcut discards.

Solution to Exercise 2.4. Two mechanisms, both consistent with the chapter, explain a *drop* in AUC after trimming $|z| > 3$ rows.

(a) *The outliers were the signal.* In credit risk the extreme tail of behaviour is disproportionately informative about default: erratic whale deposits, balance-inflation spikes, abnormal withdrawal bursts (see the pitfall in Section 2.3). Deleting $|z| > 3$ rows removes a non-random, minority-class-enriched slice of the training data — defaulters exhibit extreme behaviour more often than repayers — so the model loses precisely the discriminative region of the feature space, and its ranking quality on an untrimmed evaluation set falls.

(b) *The analyst created a training–serving mismatch.* Rows were removed from the *training* data, but the validation/test data (and, eventually, production traffic) still contain extreme rows. The model is now evaluated on a region of the input space it was never fitted on — a self-inflicted violation of the distribution-overlap requirement of Section 2.6. A subsidiary aggravating factor: on heavy-tailed transaction data the Gaussian $|z| > 3$ rule flags a large fraction of perfectly legitimate rows (the z -score’s μ and σ are themselves inflated by the tail), so the deletion is both extensive and systematically targeted at high-activity customers.

Solution to Exercise 2.5. From Eq. (2.7), the maximum maps to

$$x'_{\max} = \frac{x_{\max} - x_{\min}}{x_{\max} - x_{\min}} = 1$$

identically — the scaled value of the largest point is 1 whatever its magnitude, which is exactly why a single extreme outlier cannot be “detected” in the scaled data: it is normalised into the boundary.

Let m be the median. Its scaled value is $m' = (m - x_{\min})/(x_{\max} - x_{\min})$. Replace the maximum by $10x_{\max}$ (all other order statistics unchanged); the new scaled median is

$$m'' = \frac{m - x_{\min}}{10x_{\max} - x_{\min}}, \quad \frac{m''}{m'} = \frac{x_{\max} - x_{\min}}{10x_{\max} - x_{\min}}.$$

For $x_{\min} = 0$ this ratio is exactly $1/10$: the median's scaled value collapses by a factor of ten, and with it the entire bulk of the distribution is crushed toward 0 while the outlier alone occupies the top of the unit interval. This is the fragility that motivates the robust scaler of [Eq. \(2.8\)](#).

Solution to Exercise 2.6. Within level c , model each target as conditionally Bernoulli, $y_i | \theta_c \sim \text{Bernoulli}(\theta_c)$, and place the conjugate prior

$$\theta_c \sim \text{Beta}(\alpha, \beta), \quad \alpha = m\bar{y}, \quad \beta = m(1 - \bar{y}),$$

so that the prior mean is $\alpha/(\alpha + \beta) = \bar{y}$ (the global rate) and the prior strength — the number of pseudo-observations — is $\alpha + \beta = m$. Observing n_c rows of level c with $s_c = \sum_i y_i = n_c \bar{y}_c$ successes, conjugacy gives the posterior $\theta_c | \text{data} \sim \text{Beta}(\alpha + s_c, \beta + n_c - s_c)$, whose mean is

$$\mathbb{E}[\theta_c | \text{data}] = \frac{\alpha + s_c}{\alpha + \beta + n_c} = \frac{m\bar{y} + n_c \bar{y}_c}{m + n_c} = \text{TE}(c),$$

which is [Eq. \(2.11\)](#) exactly. The limits confirm the shrinkage behaviour: as $n_c \rightarrow 0$, $\text{TE}(c) \rightarrow \bar{y}$ (an unseen or rare level is pulled entirely to the prior), and as $n_c \rightarrow \infty$, $\text{TE}(c) \rightarrow \bar{y}_c$ (abundant evidence overwhelms the prior). The smoothing parameter m is thus interpretable as the number of global-prior pseudo-counts each level must “outweigh” before its own history dominates — and setting it too small is precisely what lets a level seen once memorise its own label ([Section 2.5.3](#)). $\square$

Solution to Exercise 2.7. *Mechanism.* Individually, a start date and an end date are weak predictors of churn. Jointly they encode the label almost perfectly at extraction time: an employee who has churned has a populated `contract_end_date` lying before the snapshot date, whereas a current employee has a null, sentinel, or future-dated value. Any model able to combine the two features (or a feature and the implicit snapshot date) reconstructs “has this contract already terminated?” — which *is* the churn label, or a near-deterministic proxy for it. The end date is a *post-outcome field*, written by the HR system as a consequence of the very event being predicted: cause 6 of [Section 2.7.2](#), hiding behind two innocuous-looking columns that pass a univariate correlation screen.

A screen for pairwise leakage. Extend the univariate screen of [Section 2.7](#) to constructed pairs: (i) for every pair of same-typed columns (two dates, two amounts), automatically generate their difference (and, for amounts, their ratio) and rank these derived features by $|\rho(\cdot, y)|$ or mutual information with the target; (ii) as a model-based variant, fit a depth-2 decision tree on each feature pair alone and flag any pair whose standalone AUC approaches the full model's. Either variant catches start/end-date leakage immediately (the difference is tenure-to-termination; the tiny tree learns “end date < snapshot”). The screen complements, but does not replace, the lineage question — *which system writes this field, and when?* — that identifies post-outcome fields even when their statistical footprint is subtle.

Solution to Exercise 2.8. Assume the hash of [Eq. \(2.10\)](#) assigns each of the K levels independently and uniformly to one of d buckets. Two given levels collide with probability $1/d$, and there are $\binom{K}{2}$ pairs, so by linearity of expectation the expected number of colliding pairs is

$$\mathbb{E}[\#\text{collisions}] = \binom{K}{2} \frac{1}{d} = \frac{K(K-1)}{2d}.$$

For $K = 10^4$ and $d = 256$:

$$\frac{10^4 \times (10^4 - 1)}{2 \times 256} = \frac{99\,990\,000}{512} \approx 1.95 \times 10^5$$

colliding pairs — with an average bucket load of $K/d \approx 39$ levels, so *every* bucket is shared. This is not necessarily fatal (tree models tolerate the induced noise, and the load is spread evenly), but it quantifies the trade-off: at this cardinality, $d = 256$ buys a fixed 256-column footprint at the price of heavy aliasing, and raising d to 2^{12} – 2^{14} reduces the expected pair collisions by one to two orders of magnitude while remaining far cheaper than a 10^4 -column one-hot encoding.

Solution to Exercise 2.9. *Leakage mechanism.* Support-line contact is not exogenous to repayment trouble. Two concrete routes: (i) borrowers who are already struggling call to ask about extensions, penalties, or restructuring *after* the loan being scored has been disbursed — so if the 7-day window is computed relative to label-observation time (or to “now” in the training warehouse) rather than to the scoring timestamp t , the feature ingests post- t information in direct violation of the as-of rule (Eq. (2.1)); (ii) worse, some inbound calls are *returns of collections outreach* — the collections team calls delinquent borrowers, who call back — making the count a laundered copy of the delinquency flag, exactly the post-outcome pattern of `days_since_collections_call` in Listing 2.11 (cause 6 of Section 2.7.2).

Redesign. Keep the legitimate signal (a customer’s baseline propensity to need support may genuinely correlate with risk) by: fixing the window to end strictly before the scoring timestamp, $[t - 7d, t)$, enforced by the feature platform’s point-in-time semantics (Section 2.8); excluding call types causally downstream of the credit process (collections callbacks, repayment-difficulty queries about an *active* loan), which requires the call-category field and its write-time lineage; and validating the rebuilt feature with the correlation screen — its association with the target should fall from “implausible” to “modest” if the leak, and not the baseline behaviour, was carrying the load.

Solution to Exercise 2.10. A five-sentence rebuttal. First, native NaN routing (Chen and Guestrin, 2016) solves only the mechanical problem of fitting a tree through incomplete rows; it does not answer the diagnostic questions — *why* is the value missing, and is the missingness itself predictive? — that Lab 2’s MCAR/MAR/MNAR audit exists to settle. Second, under MNAR the explicit indicator of Remark 2.1 is a first-class, interpretable feature whose effect can be measured, reviewed for fairness, and explained to a credit officer, whereas a learned default branch direction is an implicit, per-split artefact that cannot. Third, the pipeline must remain model-agnostic: the moment we benchmark against a regulatory logistic scorecard (Eq. (2.13)) or a neural baseline (??), models without native handling need the imputation stage anyway, and comparing models across different missing-data treatments confounds the comparison. Fourth, training–serving consistency (Section 2.8) requires that whatever missing-value semantics the model learned be reproduced exactly in the online path — a fitted, serialised imputer inside the Pipeline of Listing 2.12 is a deployable contract, while “the library handles it” is not. Fifth, most of Lab 2’s effort — parsing, validity rules, entity resolution, leakage hunting — is not imputation at all, and no amount of native NaN support dedupes a customer with three SIM cards.

Chapter 3: Classical Machine Learning and Rigorous Evaluation

Solution to Exercise 3.1. A defensible committee-minute sentence: “*Holding every other feature in the model fixed, applicants with late-night transaction activity have $e^{0.62} \approx 1.86$ times the odds of default of otherwise-identical applicants without it.*” The load-bearing words are *odds* and *holding every other feature fixed*.

The important caveat: the coefficient measures a *conditional association, not a causal effect*. It is a partial effect within this particular model and this particular preprocessing pipeline (Chapter 2): add, remove, or re-encode correlated features and the coefficient — and hence the “effect” of late-night activity — can change substantially. It licenses no intervention claim (“discouraging late-night transactions would reduce defaults” does not follow). A secondary caveat worth a footnote in the minute: an odds ratio is not a risk ratio. At the portfolio’s 3% base rate the two are numerically close ($OR \approx RR$ when the outcome is rare), but the identification should still be avoided in a document regulators may read.

Solution to Exercise 3.2. *Derivative of the sigmoid.* With $\sigma(z) = (1 + e^{-z})^{-1}$,

$$\frac{\partial \sigma}{\partial z} = \frac{e^{-z}}{(1 + e^{-z})^2} = \underbrace{\frac{1}{1 + e^{-z}}}_{\sigma(z)} \cdot \underbrace{\frac{e^{-z}}{1 + e^{-z}}}_{1 - \sigma(z)} = \sigma(z)(1 - \sigma(z)).$$

Gradient of the log loss. Write the per-example loss with $\hat{p} = \sigma(z)$, $z = \boldsymbol{\theta}^\top \mathbf{x}$:

$$\ell(\boldsymbol{\theta}) = -y \log \hat{p} - (1 - y) \log(1 - \hat{p}).$$

By the chain rule,

$$\frac{\partial \ell}{\partial z} = -\frac{y}{\hat{p}} \hat{p}(1 - \hat{p}) + \frac{1 - y}{1 - \hat{p}} \hat{p}(1 - \hat{p}) = -y(1 - \hat{p}) + (1 - y)\hat{p} = \hat{p} - y,$$

and since $\partial z / \partial \boldsymbol{\theta} = \mathbf{x}$, $\nabla_{\boldsymbol{\theta}} \ell = (\hat{p} - y) \mathbf{x}$. Averaging over the m training examples gives Eq. (3.4): $\nabla_{\boldsymbol{\theta}} J = \frac{1}{m} \sum_i (\hat{p}_i - y_i) \mathbf{x}_i$. The remarkable cancellation of the $\hat{p}(1 - \hat{p})$ factor is a signature of pairing the sigmoid with its *matched* loss (cross-entropy); the same cancellation reappears at the output layer of every neural classifier in ??.

Solution to Exercise 3.3. From Eq. (3.11), $F_\beta = (1 + \beta^2)PR / (\beta^2 P + R)$.

Limit $\beta \rightarrow 0$: the expression tends to $PR/R = P$ directly.

Limit $\beta \rightarrow \infty$: divide numerator and denominator by β^2 :

$$F_\beta = \frac{(1 + \beta^{-2})PR}{P + \beta^{-2}R} \xrightarrow{\beta \rightarrow \infty} \frac{PR}{P} = R.$$

So β interpolates between a pure-precision and a pure-recall score, which is exactly why it should be set from the cost structure.

Numerical part. With $P = 0.40$, $R = 0.75$, $\beta = 2$:

$$F_2 = \frac{(1 + 4) \cdot 0.40 \cdot 0.75}{4 \cdot 0.40 + 0.75} = \frac{1.50}{2.35} \approx 0.638,$$

noticeably above $F_1 = 2 \cdot 0.3 / 1.15 \approx 0.522$ because F_2 rewards this classifier’s recall-heavy profile.

Solution to Exercise 3.4. Fix an instance with calibrated probability p of being positive. The two actions have expected costs

$$\mathbb{E}[\text{cost} \mid \text{flag}] = p c_{TP} + (1 - p) c_{FP}, \quad \mathbb{E}[\text{cost} \mid \text{pass}] = p c_{FN} + (1 - p) c_{TN}.$$

Flagging is optimal iff $p c_{TP} + (1 - p) c_{FP} \leq p c_{FN} + (1 - p) c_{TN}$, i.e.

$$(1 - p)(c_{FP} - c_{TN}) \leq p(c_{FN} - c_{TP}).$$

Under the “reasonableness” conditions $c_{FP} > c_{TN}$ and $c_{FN} > c_{TP}$ (each error costs more than the corresponding correct decision — otherwise one action dominates and no threshold exists), both bracketed terms are positive and we may divide:

$$t^* = \frac{c_{FP} - c_{TN}}{(c_{FP} - c_{TN}) + (c_{FN} - c_{TP})}.$$

Setting $c_{TP} = c_{TN} = 0$ recovers Eq. (3.13). Only cost *differences* within each true class matter — adding a constant to a whole row of the cost matrix leaves t^* unchanged. Concretely: if every flag triggers a manual review costing KES 200 regardless of outcome ($c_{TP} = c_{FP, \text{new}} - c_{FP, \text{old}} = 200$ added to the flag column), then with the case-study numbers $t^* = (1000)/(1000+9800) \approx 0.093$: review costs push the threshold up, and the scorer should flag slightly less eagerly.

Solution to Exercise 3.5. Let the positive scores be $s_1^+, \dots, s_{m_+}^+$ and the negative scores $s_1^-, \dots, s_{m_-}^-$, and assume first that all scores are distinct. Sort all $m_+ + m_-$ instances by decreasing score and trace the empirical ROC curve from (0,0): each positive encountered moves the curve *up* by $1/m_+$, each negative moves it *right* by $1/m_-$.

Consider the horizontal step contributed by a particular negative j . Its height above the FPR axis at that moment is (number of positives already passed)/ $m_+ = \#\{i : s_i^+ > s_j^-\}/m_+$. The area of that step is therefore $\#\{i : s_i^+ > s_j^-\} / (m_+ m_-)$. Summing over all negatives,

$$\text{AUC} = \frac{1}{m_+ m_-} \sum_{j=1}^{m_-} \sum_{i=1}^{m_+} \mathbb{I}[s_i^+ > s_j^-] = \frac{U}{m_+ m_-},$$

where $U = \sum_{i,j} \mathbb{I}[s_i^+ > s_j^-]$ is precisely the Mann–Whitney U statistic of the positive-class scores. Dividing by the number of pairs turns the count into the empirical probability $P(s^+ > s^-)$ for a uniformly random (positive, negative) pair, which is Proposition 3.2.

Ties. If k_+ positives and k_- negatives share a score, the ROC convention traces a straight diagonal segment through the tied block, contributing area equal to what the pairs would contribute at $\frac{1}{2}$ each; correspondingly U is defined with $\frac{1}{2}$ per tied pair. This yields the tie-corrected statement $\text{AUC} = P(s^+ > s^-) + \frac{1}{2}P(s^+ = s^-)$ of Eq. (3.15).

Solution to Exercise 3.6. Suppose scores are independent of labels. Fix any threshold t and let $A = \{\mathbf{x} : s(\mathbf{x}) \geq t\}$ be the alert set. By independence, the label distribution *inside* A equals the marginal label distribution, so

$$\mathbb{E}[\text{Precision}(t)] = P(y = 1 \mid \mathbf{x} \in A) = \pi,$$

the prevalence — at *every* threshold, hence at every achievable recall level. The PR “curve” of a label-independent scorer is therefore the horizontal line at height π , and its area (average precision) is

$$\text{AP} = \int_0^1 \pi d(\text{recall}) = \pi.$$

On the case-study portfolio $\pi \approx 0.03$, so chance PR–AUC ≈ 0.03 . Reporting “PR–AUC = 0.31” without the baseline invites two opposite misreadings: against an implicit 0.5 anchor (borrowed from ROC intuition) it looks poor, whereas against the true chance level it is a tenfold lift. The convention of Section 3.4.3 — always state PR–AUC *together with* the prevalence — exists to force the correct reading.

Solution to Exercise 3.7. From Eq. (3.17), $\text{se} = \sqrt{r(1-r)/n_+} \leq 0.02$ requires

$$n_+ \geq \frac{r(1-r)}{\text{se}^2} = \frac{0.7 \times 0.3}{0.02^2} = \frac{0.21}{0.0004} = 525 \text{ positives.}$$

At 3% prevalence that means $525/0.03 = 17,500$ validation rows — for the validation split *alone*, before training data is even counted.

The implication is sobering and is the entire motivation of [Section 3.6](#): on a small credit portfolio (say 10,000 labelled loans in total, ~ 300 defaulters), *no single split can estimate recall to within two points*. The honest options are exactly the chapter’s toolkit: aggregate information across stratified folds ([Definition 3.7](#)), attach bootstrap intervals to whatever held-out estimate is reported ([Section 3.6.4](#)), and refuse to interpret differences smaller than the resulting uncertainty band.

Solution to Exercise 3.8. With $n_{01} = 63$ (only model B correct) and $n_{10} = 41$ (only model A correct), [Eq. \(3.19\)](#) gives

$$\chi^2 = \frac{(|63 - 41| - 1)^2}{63 + 41} = \frac{21^2}{104} = \frac{441}{104} \approx 4.24.$$

Since $4.24 > \chi^2_{1,0.95} = 3.84$, we reject at the 5% level the null hypothesis that the two models have equal error rates: model B’s advantage on the discordant cases is unlikely to be chance ($p \approx 0.04$).

The 9,800 concordant cases are irrelevant because they carry no information about the *difference* between the models: whether both are right or both are wrong on a case, that case shifts both models’ accuracies by the same amount. McNemar’s test conditions on the set of disagreements and asks only whether the 63:41 split is compatible with a fair coin — which is also why the test remains valid on heavily imbalanced test sets where accuracy itself is uninformative ([Section 3.3.1](#)).

Solution to Exercise 3.9. Applying [Eq. \(3.20\)](#) coordinate-wise with $\lambda = 0.4$:

$$(1, 0, 0) + 0.4 [(0, 1, 0) - (1, 0, 0)] = (1, 0, 0) + 0.4 (-1, 1, 0) = (0.6, 0.4, 0).$$

The result is invalid as a data point because a one-hot block encodes a *categorical* fact — the applicant’s region — and $(0.6, 0.4, 0)$ asserts membership of 60% in one region and 40% in another. No real applicant generates such a row; it violates the encoder’s sum-to-one, single-one structure, and any coefficient interpretation ([Example 3.1](#)) built on “the region-A indicator” silently breaks.

SMOTE-NC’s repair. SMOTE-NC interpolates only the *numeric* coordinates via [Eq. \(3.20\)](#), and sets each *categorical* feature of the synthetic point to the majority value of that feature among the k minority-class nearest neighbours (with the categorical features also entering the distance computation through a fixed penalty term). The synthetic row is then a legal row: valid one-hot blocks, interpolated numerics.

Residual risk. The synthetic point marries numeric values interpolated between two (possibly different-region) borrowers with a categorical value voted from the neighbourhood, so it can realise *combinations* of features that never co-occur in reality — e.g. transaction magnitudes typical of Nairobi CBD attached to a rural-county label. The class-conditional joint distribution is thereby distorted, and, as [Section 3.7.2](#) already warned for plain SMOTE, the altered class prior means the resulting scores are not calibrated: recalibrate ([Section 3.5](#)) before applying [Proposition 3.1](#).

Solution to Exercise 3.10. Let $w = c_{FN}/c_{FP}$ and suppose training re-weights each positive example by w (equivalently, imagine each positive duplicated w times). For a scorer that is calibrated *on the re-weighted distribution*, the probability it reports is the re-weighted posterior. If p denotes the true (original-distribution) posterior $P(y = 1 \mid \mathbf{x})$, re-weighting the positive class by w transforms it to

$$p_w = \frac{wp}{wp + (1 - p)},$$

since the class-conditional densities are untouched and only the effective prior odds are multiplied by w . Thresholding the weighted model at 0.5 flags iff

$$p_w \geq \frac{1}{2} \iff wp \geq 1 - p \iff p \geq \frac{1}{1+w} = \frac{c_{FP}}{c_{FP} + c_{FN}} = t^*,$$

which is exactly the unweighted model thresholded at t^* of Eq. (3.13). This is Elkan (2001)’s theorem: for Bayes-optimal (calibrated) scorers, cost-proportionate re-weighting and threshold moving induce the *same* decision rule.

Why try threshold moving first, then? Because the equivalence is exact only in the idealised limit, and every practical consideration favours the decision-level route. (i) It is *free and reversible*: no retraining, and the cost matrix can be revised — or differ across products — while one model serves them all, each business line simply reading a different operating point off the same score. (ii) It *preserves calibration*: re-weighting deliberately distorts the class prior, so the weighted model’s outputs are no longer probabilities on production traffic and must be recalibrated before anyone prices risk with them. (iii) With finite samples and misspecified models the two routes genuinely diverge, and re-weighting can degrade the ranking it was meant to help. (iv) It is *auditable*: “we score with model M and decline above 7.4% calibrated risk, derived from this cost matrix” is a one-line policy a regulator can inspect. Escalate to weighting or resampling only when the evidence says the *ranking itself* — not the decision rule — is the weak link (Section 3.7.1).

Bibliography

- Bishop, C. M. and Bishop, H. (2024). *Deep Learning: Foundations and Concepts*. Springer.
- Bjorkegren, D. and Grissen, D. (2020). Behavior revealed in mobile phone usage prediction credit repayment. In *The World Bank Economic Review*, volume 34.
- Box, G. E. P. and Cox, D. R. (1964). An analysis of transformations. *Journal of the Royal Statistical Society: Series B*, 26(2):211–243.
- Breiman, L. (2001). Statistical modeling: The two cultures. *Statistical Science*, 16(3):199–231.
- Brier, G. W. (1950). Verification of forecasts expressed in terms of probability. *Monthly Weather Review*, 78(1):1–3.
- Chawla, N. V., Bowyer, K. W., Hall, L., and Kegelmeyer, W. P. (2002). SMOTE: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research*, 16:321–357.
- Chen, T. and Guestrin, C. (2016). XGBoost: A scalable tree boosting system. In *Proc. 22nd ACM SIGKDD*, pages 785–794.
- Davis, J. and Goadrich, M. (2006). The relationship between precision-recall and ROC curves. In *Proceedings of the 23rd International Conference on Machine Learning (ICML)*, pages 233–240.
- Dietterich, T. G. (1998). Approximate statistical tests for comparing supervised classification learning algorithms. *Neural Computation*, 10(7):1895–1923.
- Elkan, C. (2001). The foundations of cost-sensitive learning. In *Proceedings of the 17th International Joint Conference Artificial Intelligence (IJCAI)*, pages 973–978.
- Fawcett, T. (2006). An introduction to ROC analysis. *Pattern Recognition Letters*, 27(8):861–874.
- Géron, A. (2022). *Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow*. O’Reilly Media, 3rd edition.
- Huyen, C. (2022). *Designing Machine Learning Systems*. O’Reilly Media.
- Kaufman, S., Rosset, S., Perlich, C., and Stitelman, O. (2012). Leakage in data mining: Formulation, detection, and avoidance. *ACM TKDD*, 6(4).
- Liu, F. T., Ting, K. M., and Zhou, Z.-H. (2008). Isolation forest. In *Eighth IEEE International Conference on Data Mining*, pages 413–422.
- Lundberg, S. M. and Lee, S.-I. (2017). A unified approach to interpreting model predictions. In *Advances in Neural Information Processing Systems*, volume 30.

- Micci-Barreca, D. (2001). A preprocessing scheme for high-cardinality categorical attributes in classification and prediction problems. *ACM SIGKDD Explorations Newsletter*, 3(1):27–32.
- Niculescu-Mizil, A. and Caruana, R. (2005). Predicting good probabilities with supervised learning. In *Proceedings of the 22nd International Conference on Machine Learning (ICML)*, pages 625–632.
- Platt, J. C. (1999). Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods. In *Advances in Large Margin Classifiers*, pages 61–74. MIT Press.
- Rubin, D. B. (1976). Inference and missing data. *Biometrika*, 63(3):581–592.
- Sculley, D., Holt, G., Golovin, D., Davydov, E., Phillips, T., Ebner, D., Chaudhary, V., Young, M., Crespo, J.-F., and Dennison, D. (2015). Hidden technical debt in machine learning systems. In *Advances in Neural Information Processing Systems 28*.
- Siddiqi, N. (2017). *Intelligent Credit Scoring*. Wiley, 2nd edition.
- Yeo, I.-K. and Johnson, R. A. (2000). A new family of power transformations to improve normality or symmetry. *Biometrika*, 87(4):954–959.